Dx Labs: AI for Institutional Transformation in UP

Source: ovpdx.up.edu.ph

GOAL

To ease participants into programming.

GOAL

To ease participants into programming.

NOTE:

- We don’t need to learn everything.
- We don’t need to code from scratch.

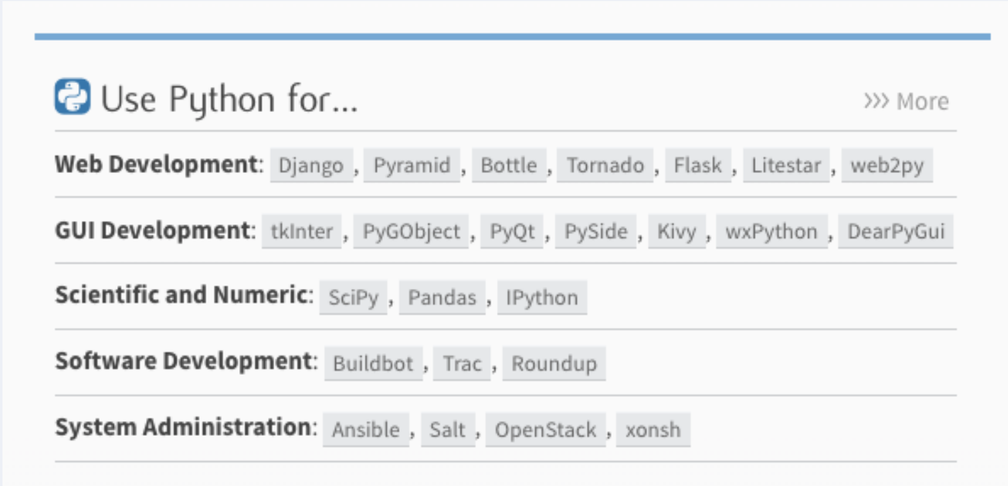
SPECIFIC GOALS

By the end of this session, the participants will:

- understand how to use AI coding assistant
- write basic Python code

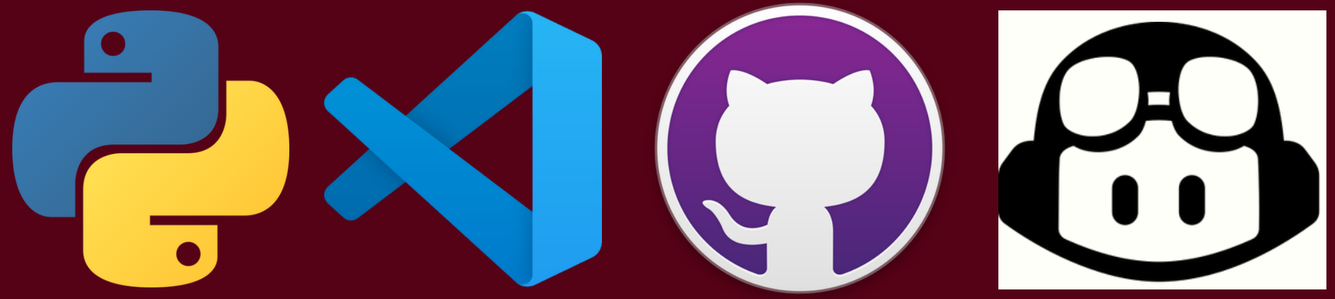
WHY PYTHON?

- One of the most popular programming language
- Simple syntax: easy to learn
- open source
- general-purpose
- great interactive environment



Source: https://www.python.org/

SETTING UP THE DEVELOPMENT ENVIRONMENT



SETTING UP OUR TOOLS

- Refer to our pre-workshop setup guide



🧰 Pre-Workshop Setup Guide

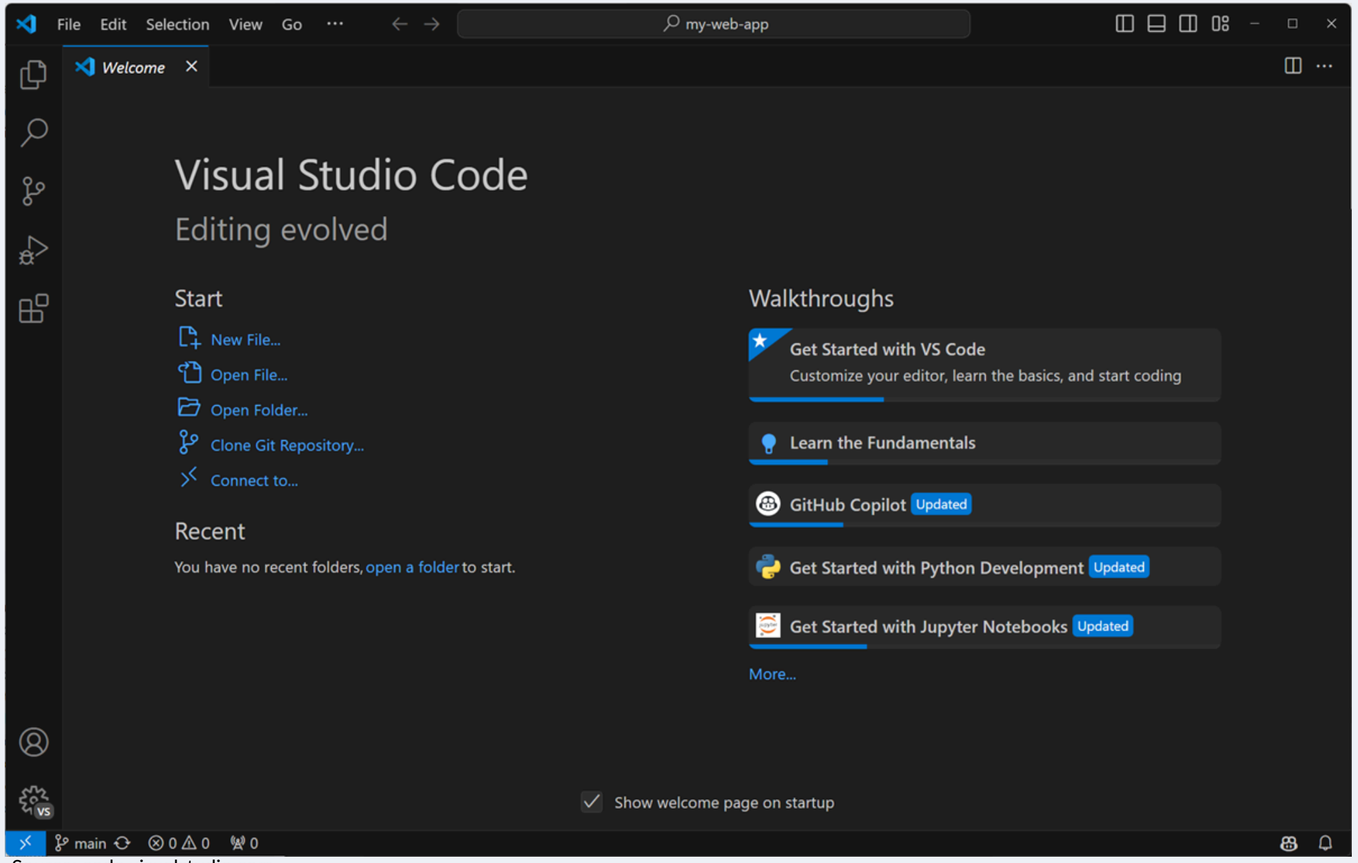
Installing Python 3, Visual Studio Code (VS Code) and Github
(For both Windows and macOS users)

This guide will help you get ready for our Python for Administrative Staff Workshop. Please follow these steps before the training session so you're ready to start coding right away!

Before proceeding, the participants are assumed to have successfully installed the needed tools following the pre-workshop setup guide

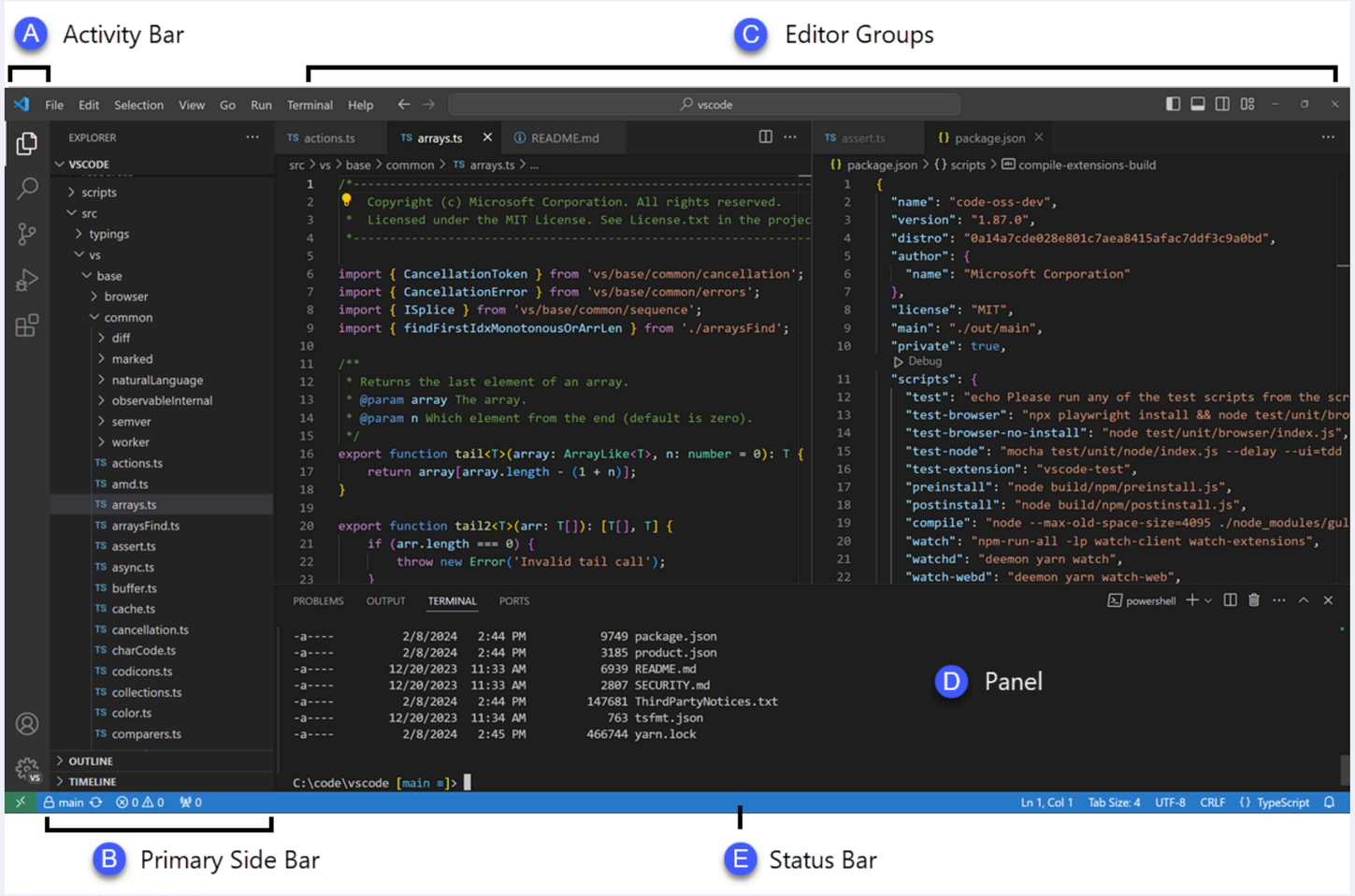
MICROSOFT VISUAL STUDIO CODE

- This is the main app we will use to write, edit and run our Python codes



Source: code.visualstudio.com

MICROSOFT VISUAL STUDIO CODE



Source: code.visualstudio.com

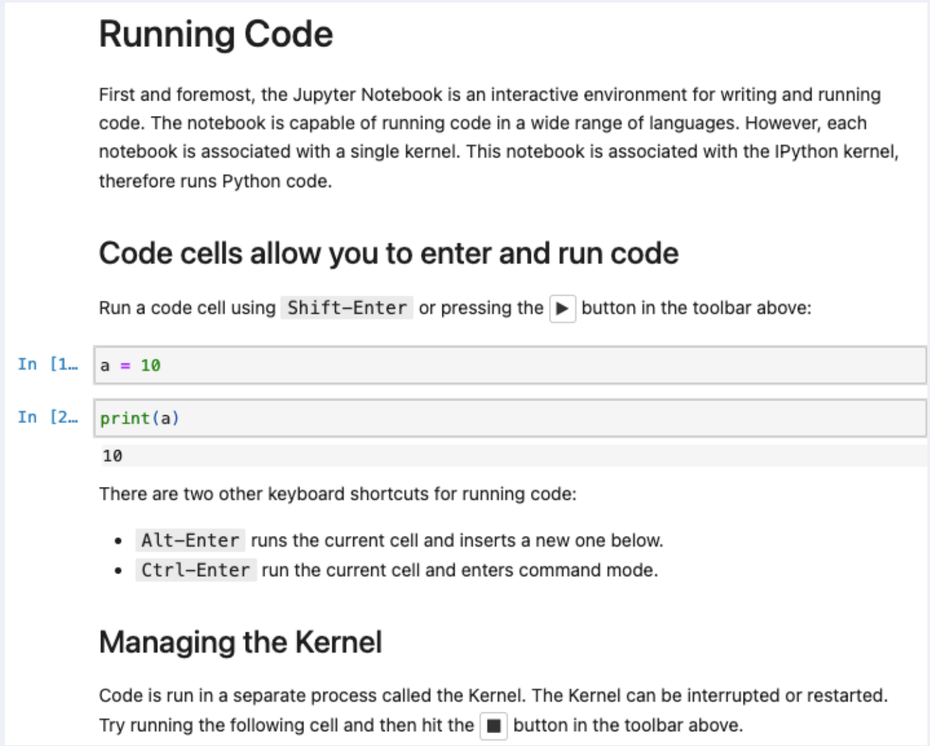
MICROSOFT VISUAL STUDIO CODE



1. Launch VS Code.
2. Create a new folder on your desktop, e.g. **DxLabs_Session01**
3. Inside VS Code, open this folder (**File --> Open Folder**).
4. Create a new file named **test_notebook.ipynb** (Python Notebook).

INTRODUCTION TO PYTHON NOTEBOOK

- rich text + runnable code
- ideal for documentation and sharing



Running Code

First and foremost, the Jupyter Notebook is an interactive environment for writing and running code. The notebook is capable of running code in a wide range of languages. However, each notebook is associated with a single kernel. This notebook is associated with the IPython kernel, therefore runs Python code.

Code cells allow you to enter and run code

Run a code cell using `Shift-Enter` or pressing the ▶ button in the toolbar above:

In [1…
```
a = 10
```

In [2…
```
print(a)
```

```
10
```

There are two other keyboard shortcuts for running code:

- `Alt-Enter` runs the current cell and inserts a new one below.
- `Ctrl-Enter` run the current cell and enters command mode.

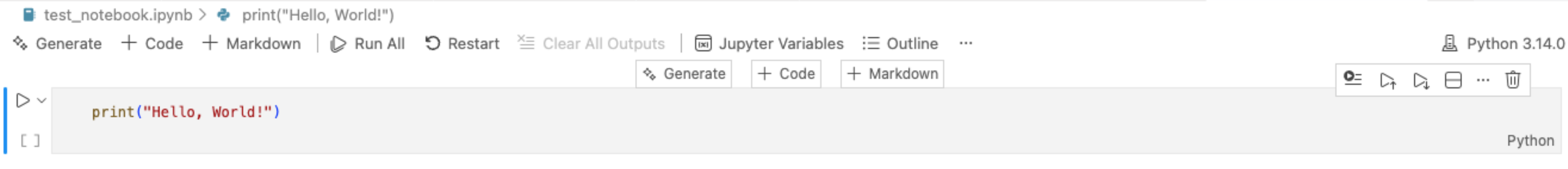
Managing the Kernel

Code is run in a separate process called the Kernel. The Kernel can be interrupted or restarted. Try running the following cell and then hit the ■ button in the toolbar above.

Source: https://www.nbviewer.org/

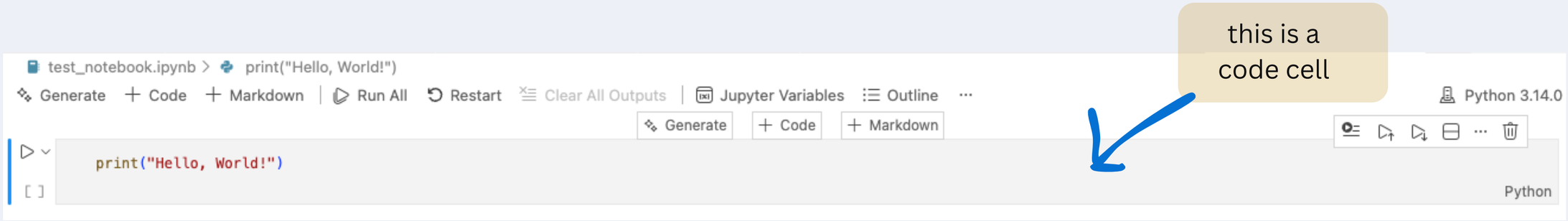
INTRODUCTION TO PYTHON NOTEBOOK

1. In the VS Code Editor, click + Code.
2. In the resulting cell, type the ff:
    - print(“Hello, World!”)
3. Press ▶ Run Cell (or Shift+ENTER) to see the result.



WITHIN PYTHON NOTEBOOK

- A cell is a place where we can write in our code or text.
- Two main types of cells:
  - Code cell
  - Markdown cell



WITHIN PYTHON NOTEBOOK

Raw markdown cell

```
# This is a header (level 1)
## This is a header (level 2)

For **bold** text

For *italicized text*

For `bulleted list`:
1. item 1
2. item 2
```

Rendered markdown cell (after pressing SHIFT+ENTER)

This is a header (level 1)

This is a header (level 2)

For **bold** text

For *italicized text*

For `bulleted list`:

1. item 1
2. item 2

To know more about markdown: https://www.markdownguide.org/cheat-sheet/

WITHIN PYTHON NOTEBOOK

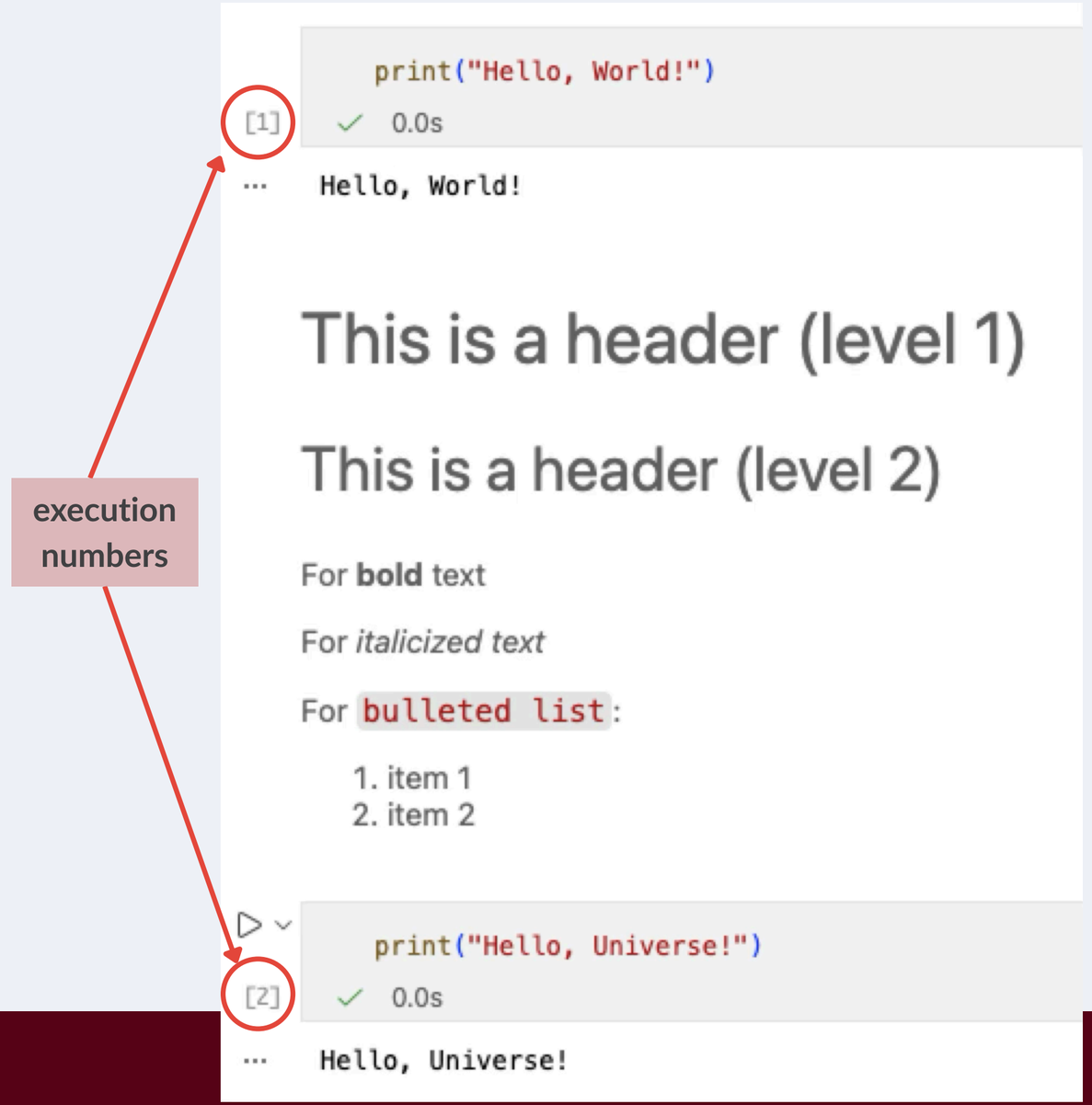
- Two types of mode:
  - Edit mode - we can type in our codes or markdown text.
  - Command mode - perform commands like adding a cell above or below an active cell, changing the type of cell, etc
- Command to Edit
  - Click inside the cell
  - Or Click on a cell then press ENTER
- Edit to Command
  - Press ESC

EDIT MODE



WITHIN PYTHON NOTEBOOK

- Code cells can be executed multiple times and in any order
- An execution number is associated to each executed code cell
  This is used to remember which code cells are executed first.
- When a code cell is not yet executed, it will have no execution number.

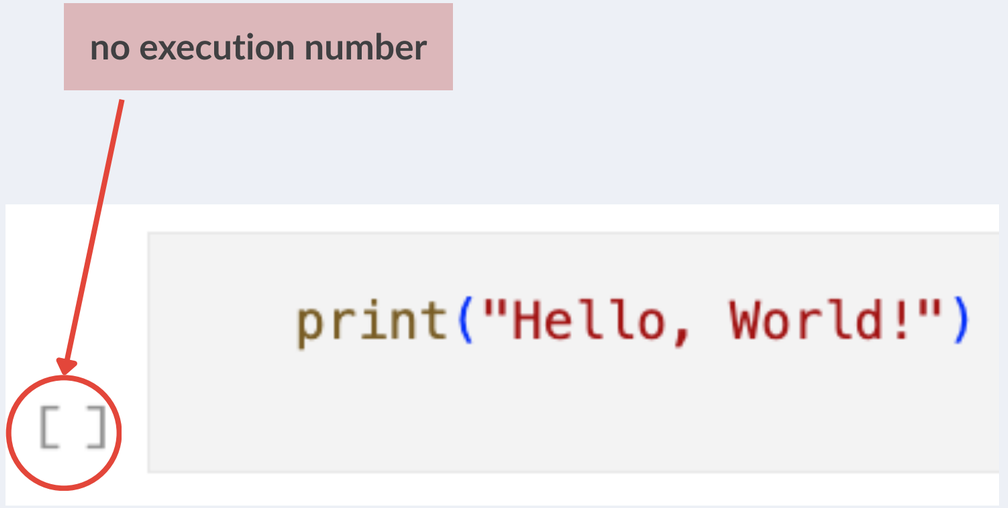


WITHIN PYTHON NOTEBOOK

- Code cells can be executed multiple times and in any order

- An execution number is associated to each executed code cell
  This is used to remember which code cells are executed first.

- When a code cell is not yet executed, it will have no execution number.



WITHIN PYTHON NOTEBOOK

Python notebook specific options



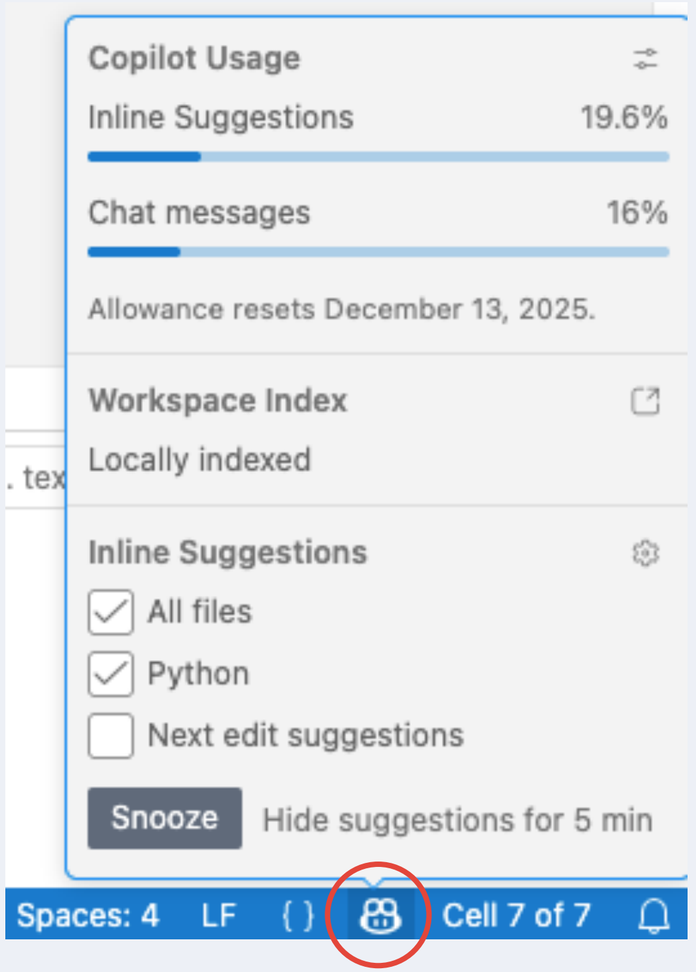


INTRODUCING COPILOT

- An AI tool that suggests code as you type
- Built on large AI models trained on public code
- Auto-complete lines and entire functions
- Generate code from comments or prompts
- Recommend fixes and improvements

INTRODUCING COPILOT

Verify your Copilot Usage by clicking the Copilot icon in the status bar

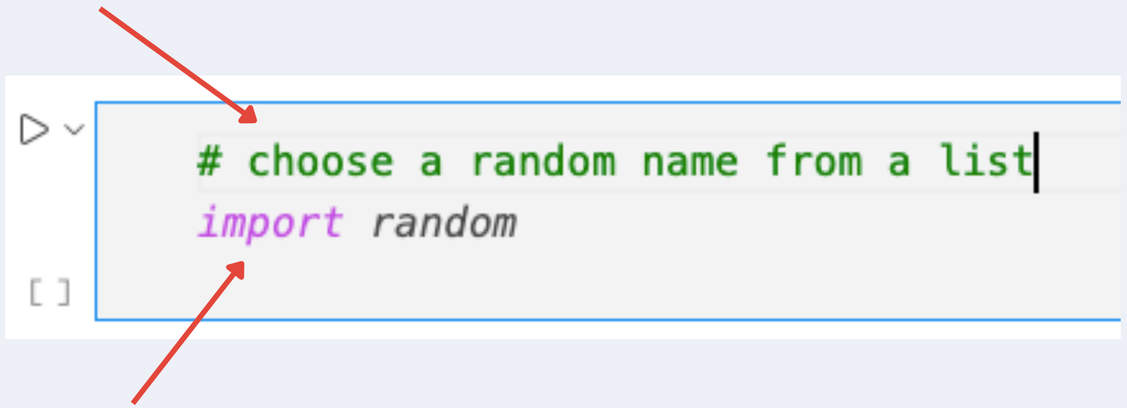




INTRODUCING COPILOT

via inline suggestions

this is our code



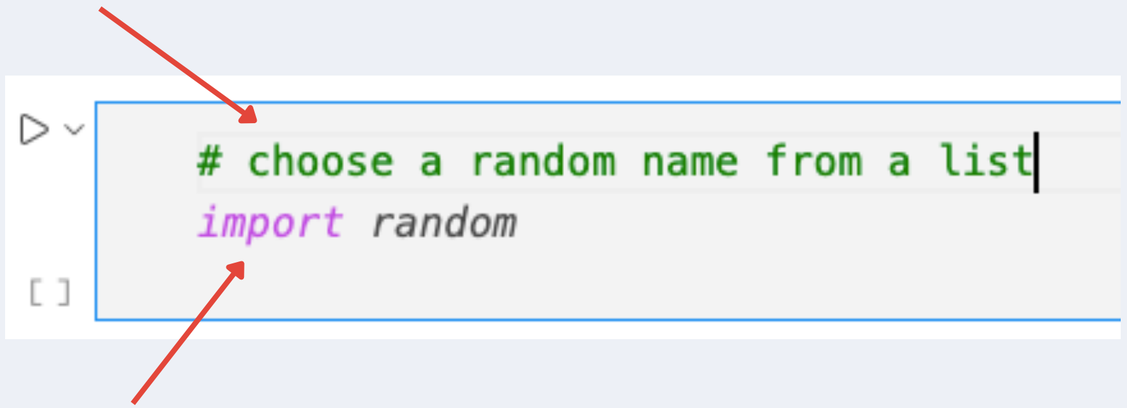
this is a suggestion
also called *ghost code*



INTRODUCING COPILOT

via inline suggestions

this is our code



this is a suggestion
also called *ghost code*

Press TAB to accept
Press ESC to reject



INTRODUCING COPILOT via inline suggestions

This is a sample code suggestion

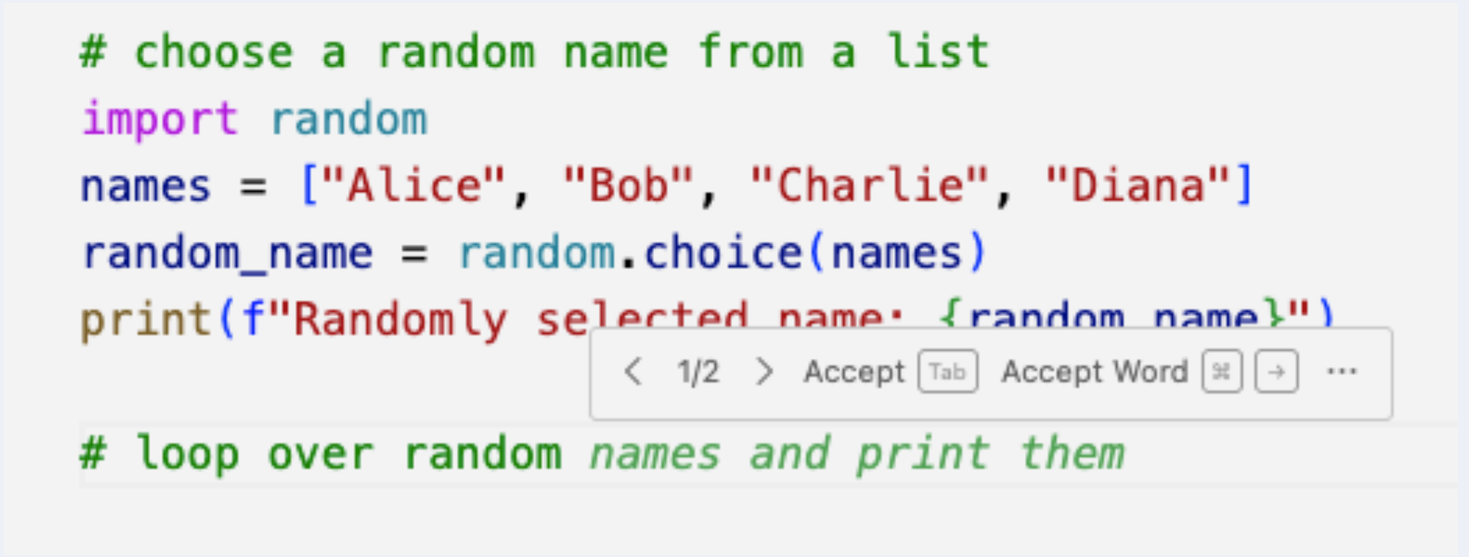
```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

Note!
You might see different suggestions because large language models are nondeterministic.

INTRODUCING COPILOT

via inline suggestions

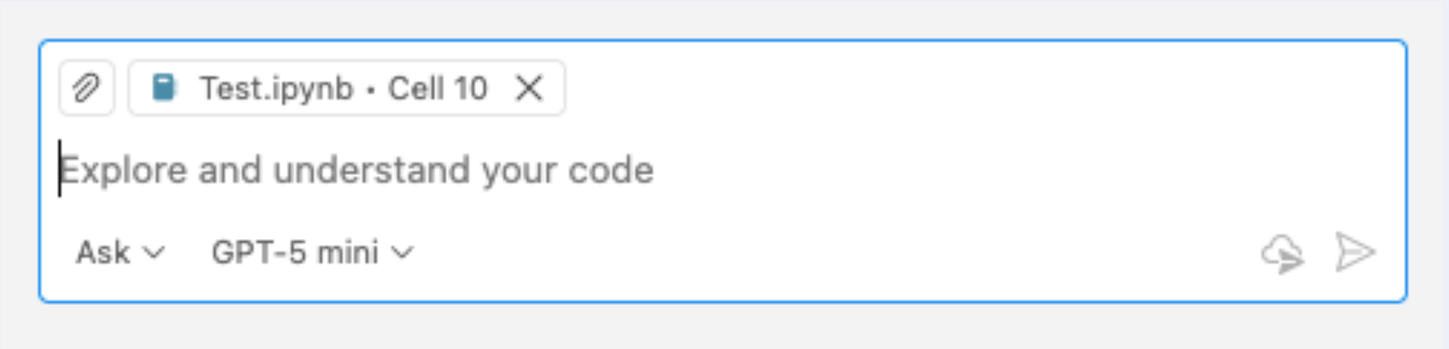
If you see multiple suggestions, hover over the ghost text to see navigation controls, or use ALT+] and ALT+[to cycle through options.



INTRODUCING COPILOT via Chat

Open the Chat view by pressing SHIFT+CMD+I or by selecting the chat icon in the VS Code title bar.

The Chat view enables you to have an ongoing conversation with the AI, making it easier to refine your requests and get better results.





INTRODUCING COPILOT

via inline chat

We can also use copilot for targeted improvements to specific code sections within a file



INTRODUCING COPILOT

via inline chat

Highlight the code section you want to improve, then press CMD+I to access the inline chat

```
# choose a random name from a list
```

Ask or edit in context

Auto

```
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

INTRODUCING COPILOT

via inline chat

the phrase we typed in
the inline chat

highlighted in green are
the suggested code edit



INTRODUCING COPILOT

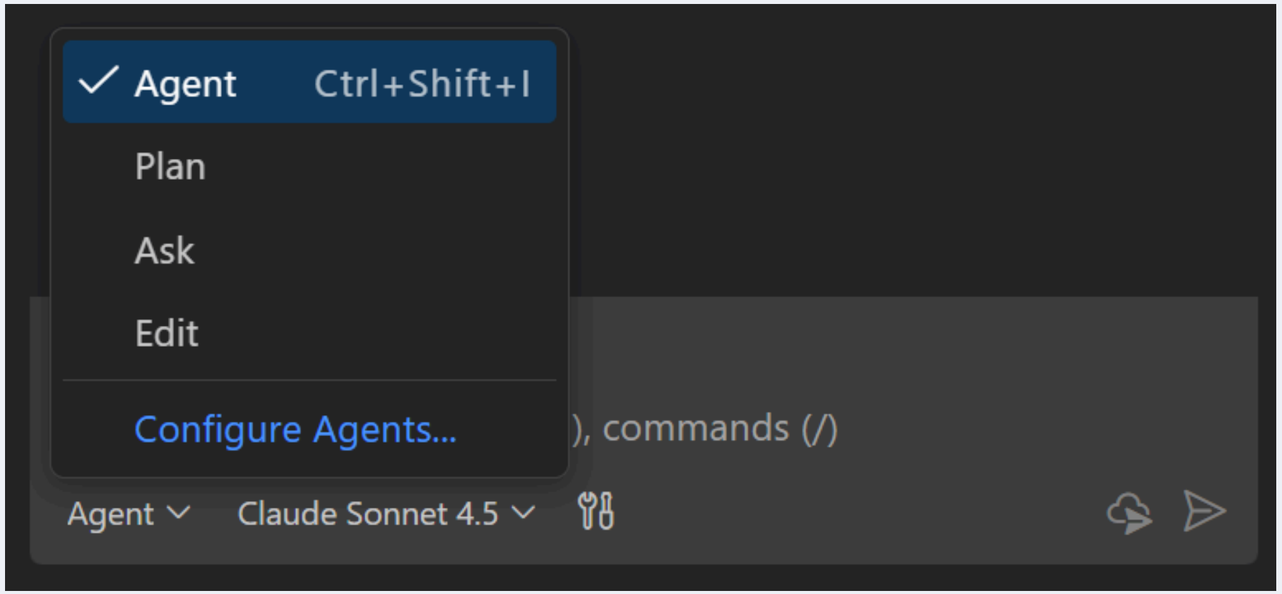
via inline chat

press Accept to adopt the code edit



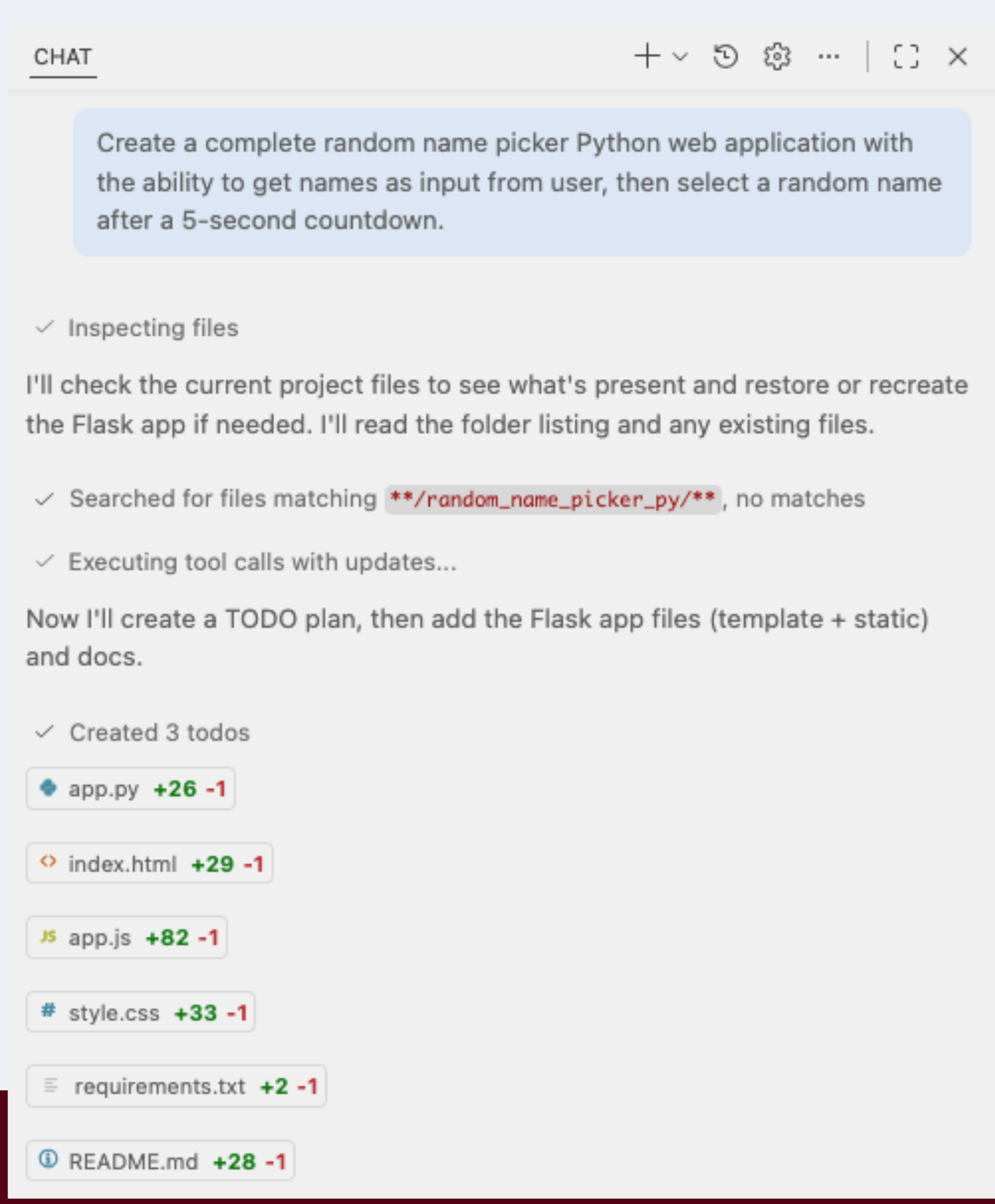
INTRODUCING COPILOT via agent

Agents are VS Code's strongest AI tool, autonomously planning and applying complex changes across files from a simple prompt.



INTRODUCING COPILOT

Agents are VS Code's strongest AI tool, autonomously planning and applying complex changes across files from a simple prompt.



SUMMARY (SO FAR)

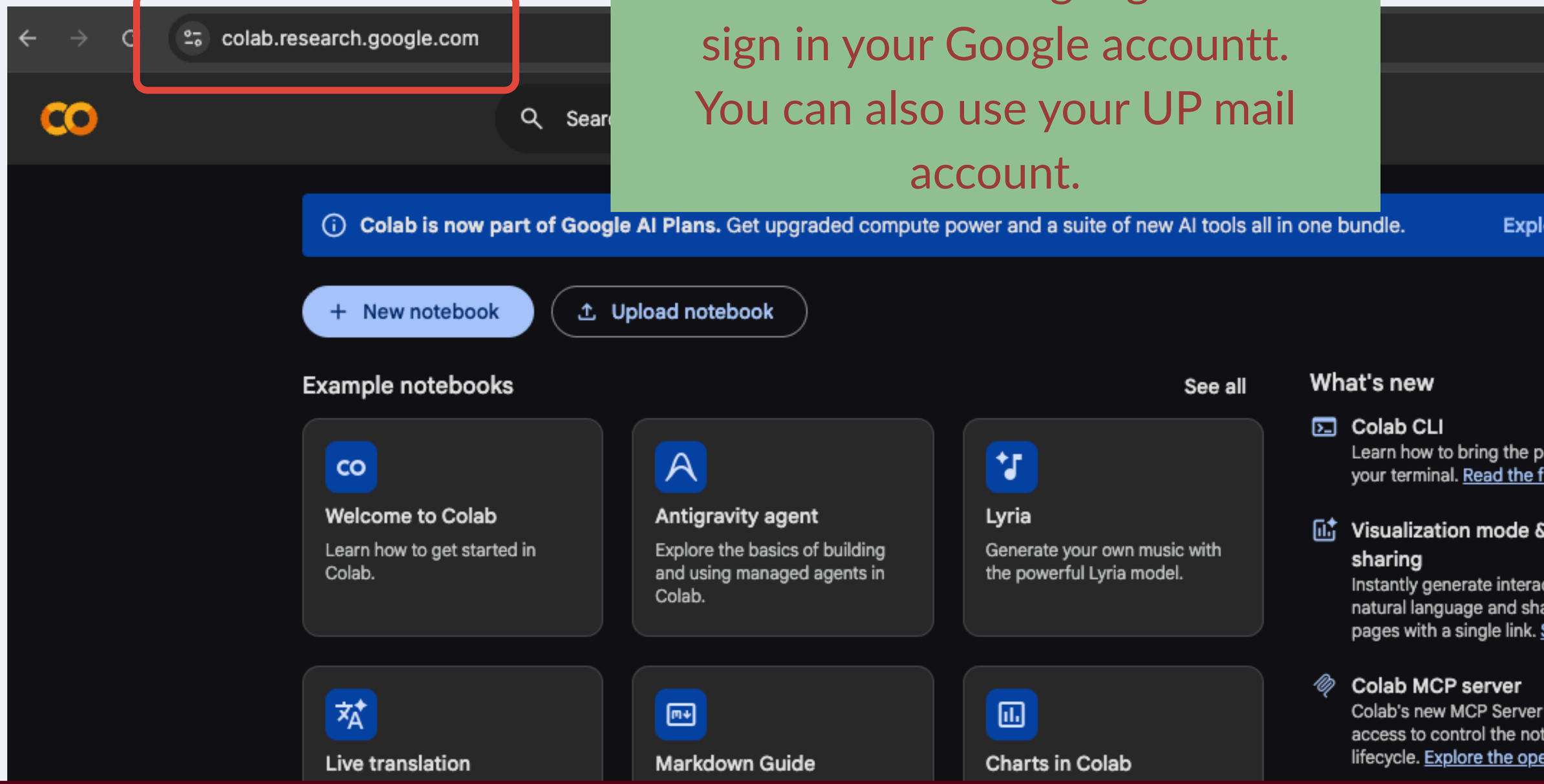
We learned about:

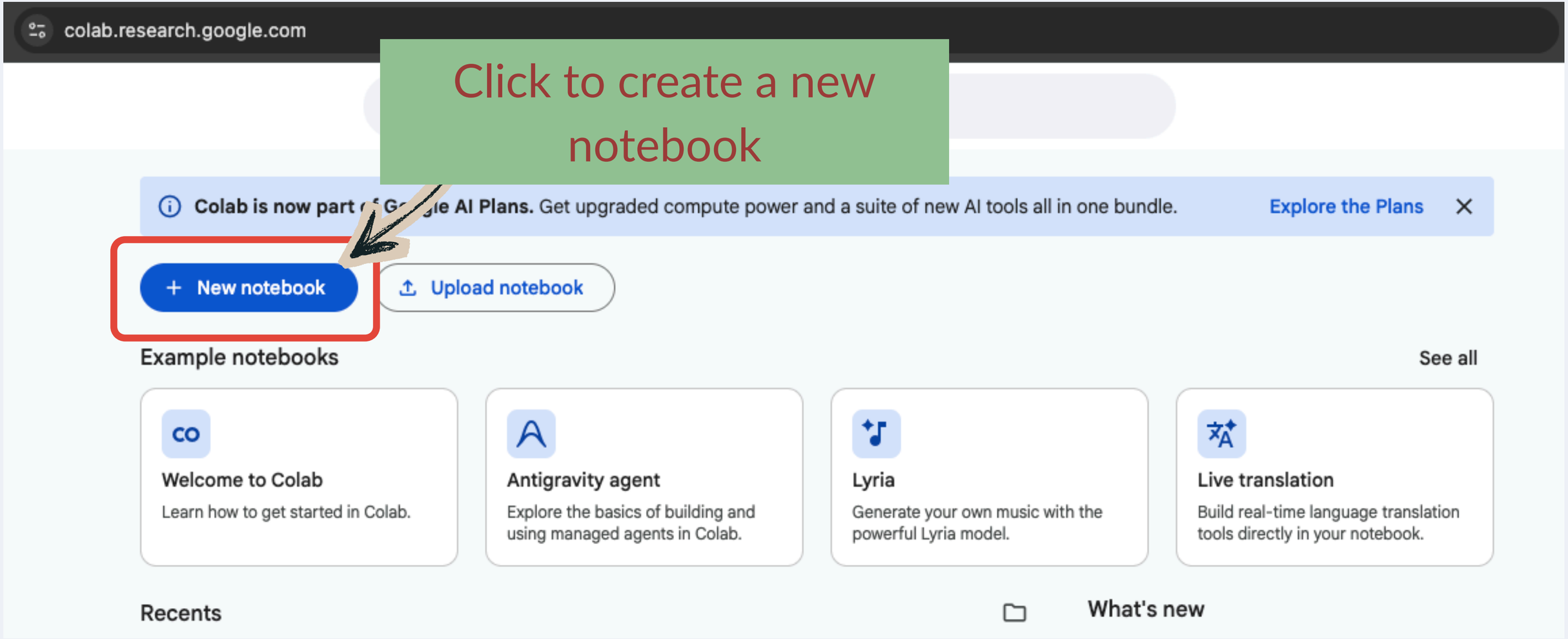
- Basic navigation on Visual Studio Code
- How to work on Jupyter notebooks
- How to use Github Copilot

AN ONLINE TOOL

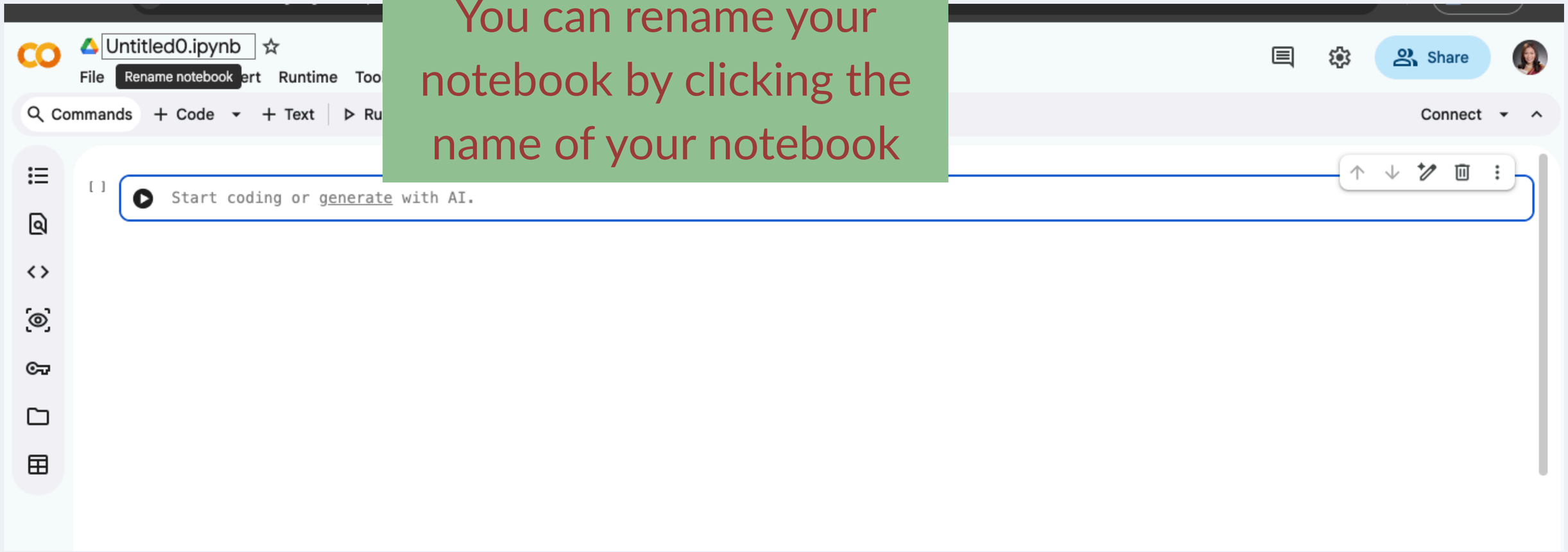
Google Colab + Gemini

Go to colab.research.google.com and sign in your Google accountt.
You can also use your UP mail account.

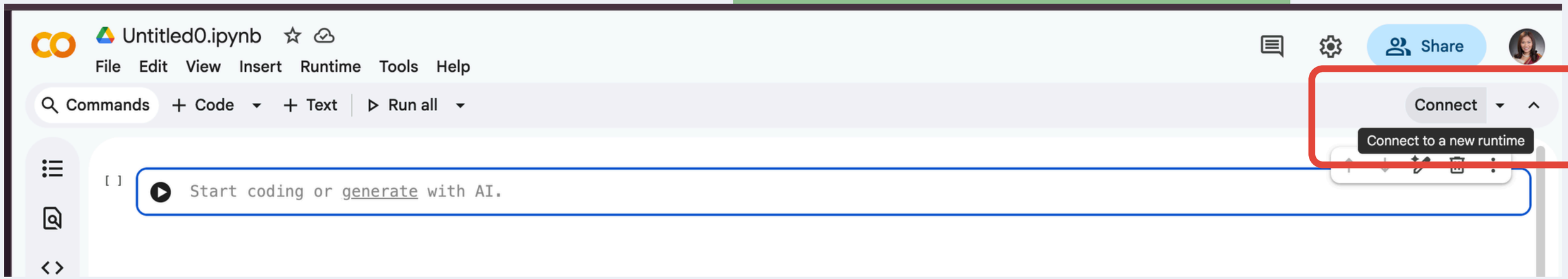


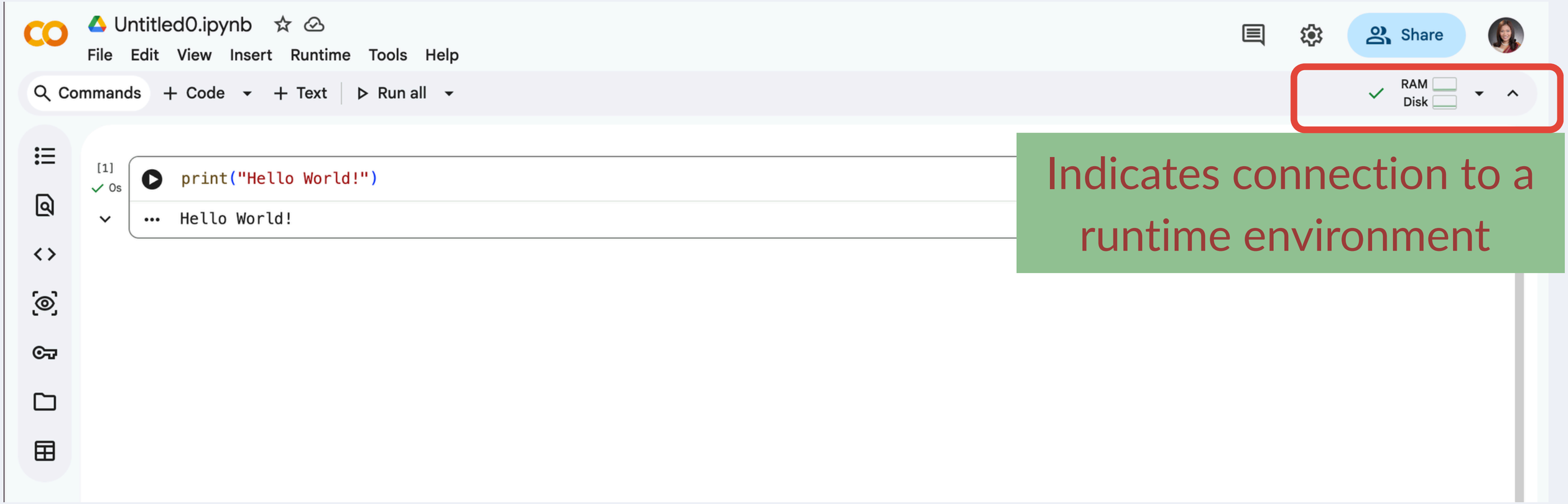

colab.research.google.com
Click to create a new notebook
Colab is now part of Google AI Plans. Get upgraded compute power and a suite of new AI tools all in one bundle.
Explore the Plans
New notebook
Upload notebook
Example notebooks
See all
Welcome to Colab
Learn how to get started in Colab.
Antigravity agent
Explore the basics of building and using managed agents in Colab.
Lyria
Generate your own music with the powerful Lyria model.
Live translation
Build real-time language translation tools directly in your notebook.
Recents
What's new

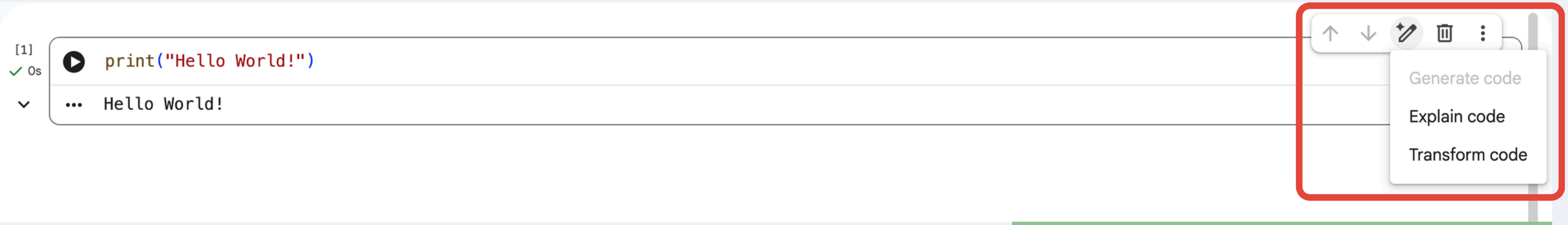
You can rename your notebook by clicking the name of your notebook



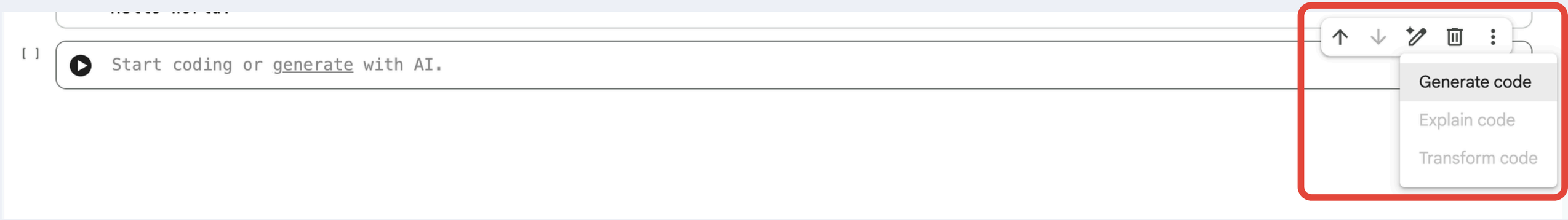
Need to connect to a runtime environment to run Python codes




Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
[1]
0s
print("Hello World!")
Hello World!
Indicates connection to a runtime environment



available AI features for an existing code



feature to generate code from prompt

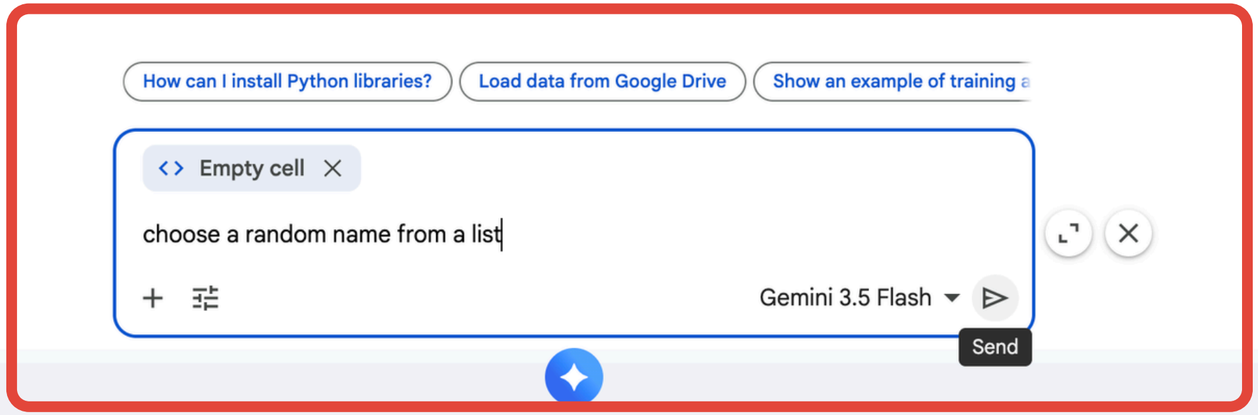
[1]
✓ 0s

```
print("Hello World!")
```

```
Hello World!
```

[ ]

Start coding or generate with AI.



Variables Terminal

✓ 10:29 AM Python 3

Gemini

```
+import random
+
+names = ["Alice", "Bob", "Charlie", "Diana", "Ethan"]
+random_name = random.choice(names)
+print(f"Selected name: {random_name}")
```

choose a random name from a list

Empty cell

Here is the code to select a random name from a list using the `random.choice` function.

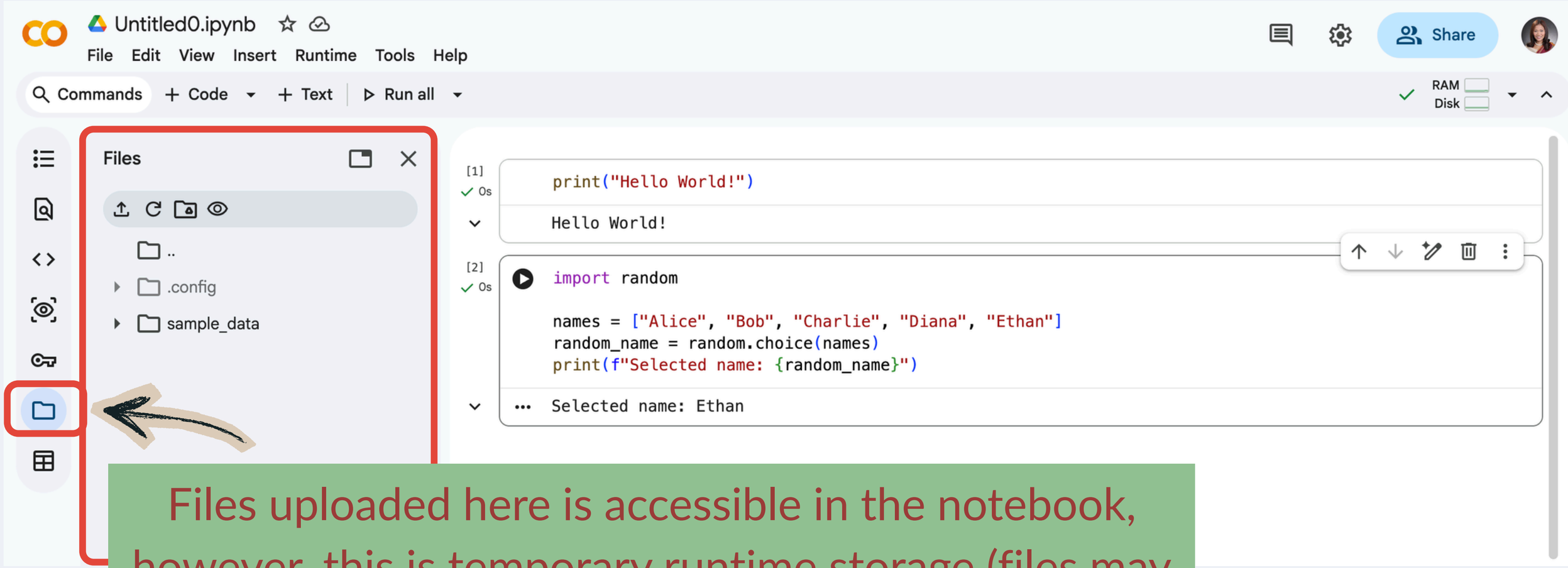
Accept & Run Accept Cancel

What can I help you build?

Gemini 3.5 Flash 18%

Gemini can make mistakes so double-check it and use code with caution. Learn more

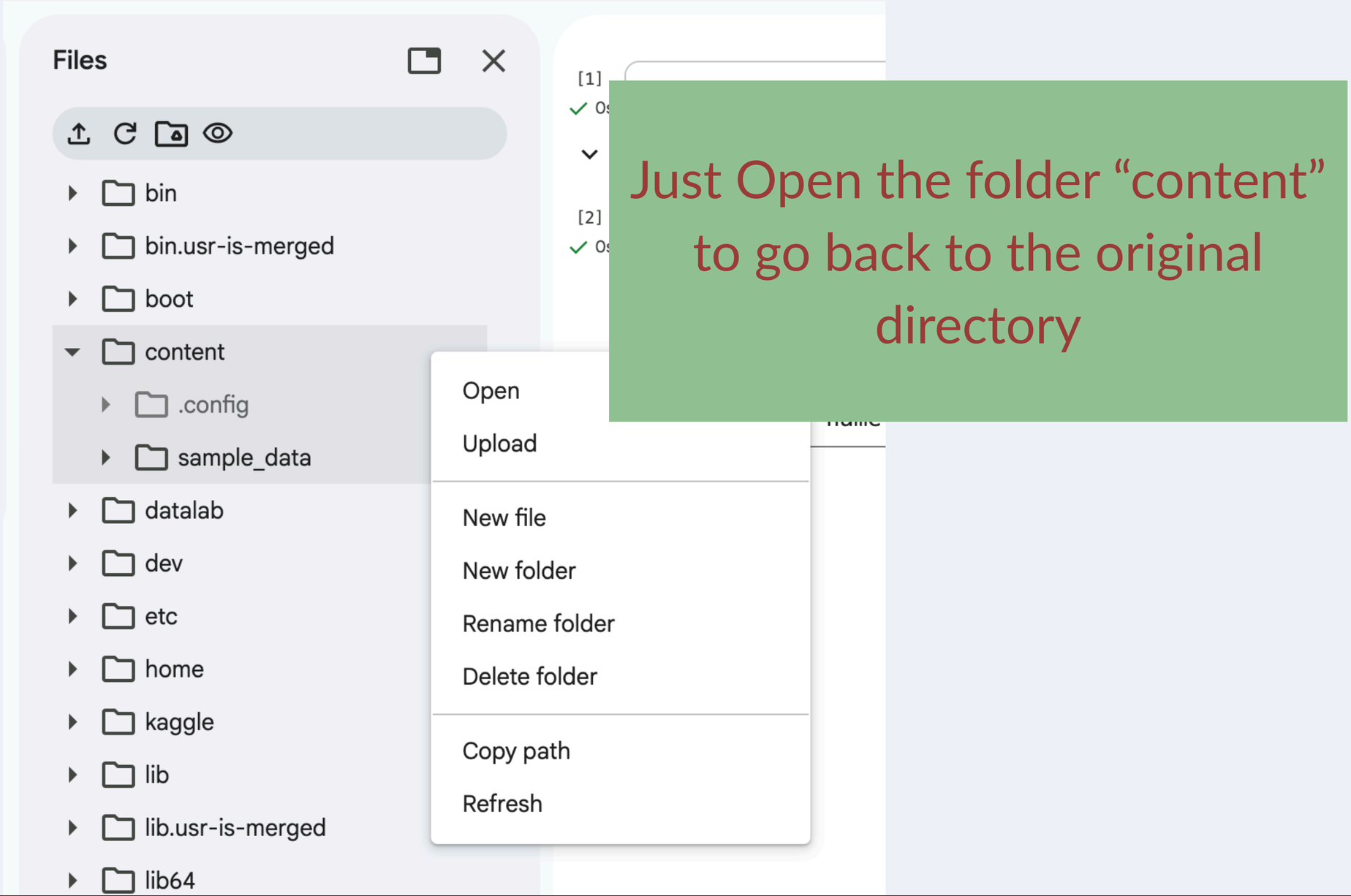
Variables Terminal 10:29 AM Python 3



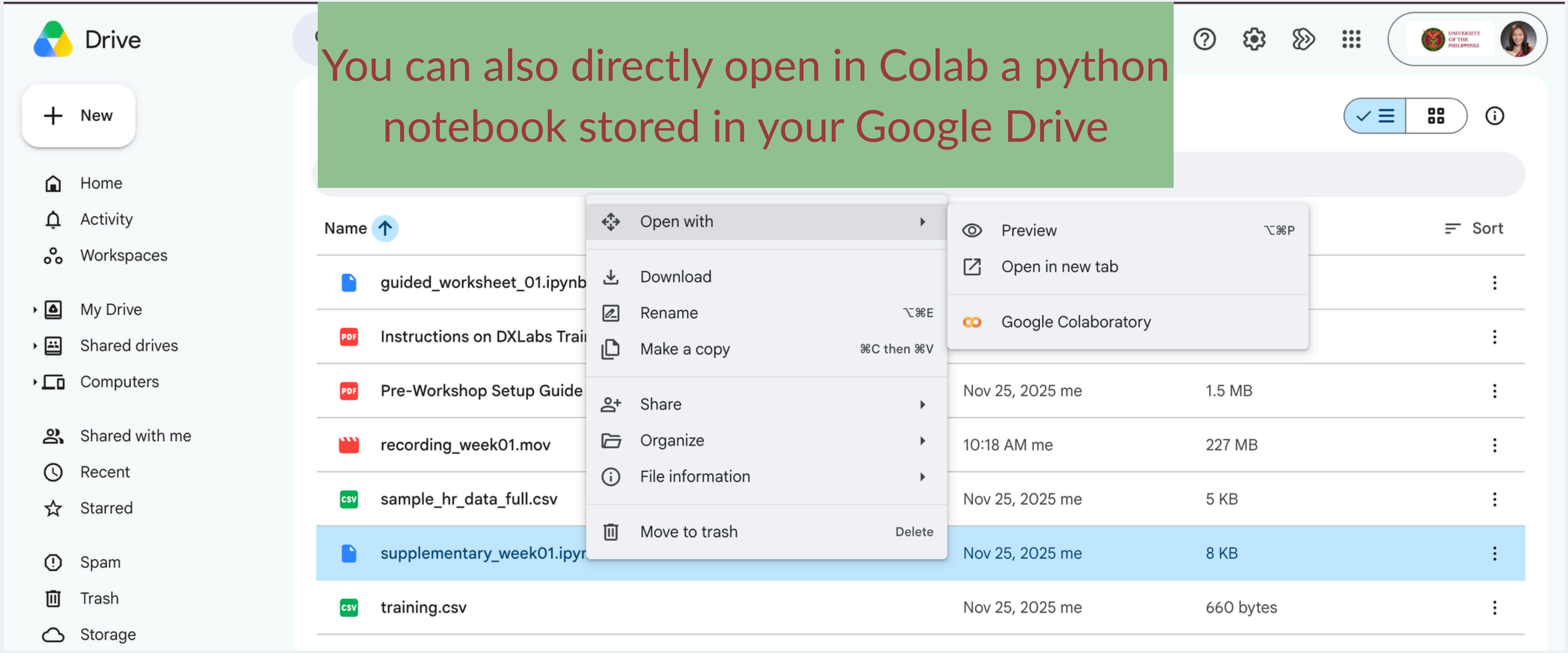
Files uploaded here is accessible in the notebook, however, this is temporary runtime storage (files may disappear when runtime disconnects or is deleted)


Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
Files
..
.config
sample_data
[1]
0s
print("Hello World!")
Hello World!
[2]
0s
import random
names = ["Alice", "Bob", "Charlie", "Diana", "Ethan"]
If you happened to click this folder with two dots, it will lead you to a parent directory


Untitled0.ipynb
File Edit View Insert Runtime Tools Help
Share
Commands + Code + Text Run all
RAM
Disk
Files
..
.config
sample_data
[1]
0s
print("Hello World!")
Hello World!
[2]
0s
import random
names = ["Alice", "Bob", "Charlie", "Diana", "Ethan"]
If you happened to click this folder with two dots (“..”), it will lead you to a parent directory


Files
bin
bin.usr-is-merged
boot
content
.config
sample_data
datalab
dev
etc
home
kaggle
lib
lib.usr-is-merged
lib64
Open
Upload
New file
New folder
Rename folder
Delete folder
Copy path
Refresh
[1]
[2]
Just Open the folder “content” to go back to the original directory

You can also directly open in Colab a python notebook stored in your Google Drive



SUMMARY

We learned about:

- Basic navigation on Visual Studio Code
- How to work on Jupyter notebooks
- How to use Github Copilot
- Explored an alternative online tool for AI-assisted coding

PYTHON BASICS

OBJECTIVE: Learn how Python syntax works through simple examples

HELLO WORLD

- print()
  print the output of whatever is inside the open and close parentheses

- “Hello, World”
  and example of a data type called a **string**.



HELLO WORLD

- Python’s way to recognize plain text
- Any characters enclosed in quotations (either single or double quotation) will be treated by Python as a string.



HELLO WORLD

- What happens if we mistyped the code?

```
print("Hello, World!"
```
[5] ⊗ 0.0s

```
  Cell In[5], line 1
    print("Hello, World!"
                          ^
_IncompleteInputError: incomplete input
```

```
print(Hello, World!)
```
[8] ⊗ 0.0s

```
  Cell In[8], line 1
    print(Hello, World!)
                       ^
SyntaxError: invalid syntax
```

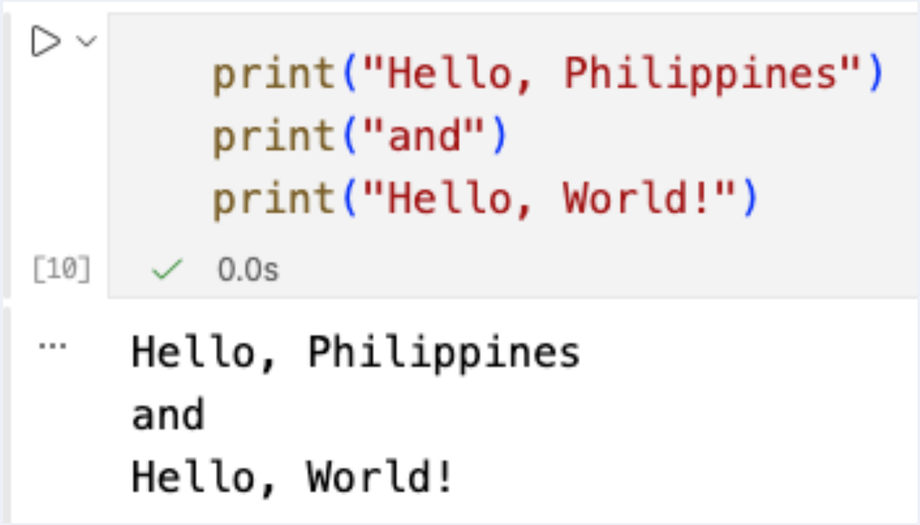
```
prin("Hello, World!")
```
[7] ⊗ 0.1s

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[7], line 1
----> 1 prin("Hello, World!")

NameError: name 'prin' is not defined
```

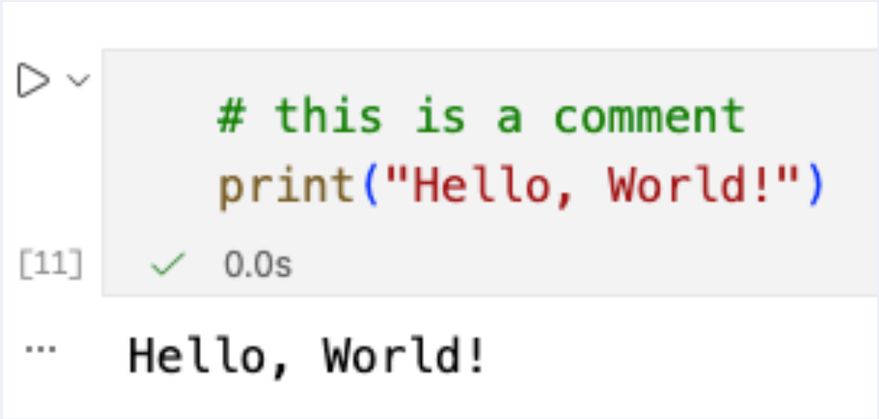
HELLO WORLD

- We can have multiple commands in one code cell.



COMMENTS

- starts with #
- No effect on the execution of codes



WORKING ON FORMULAS

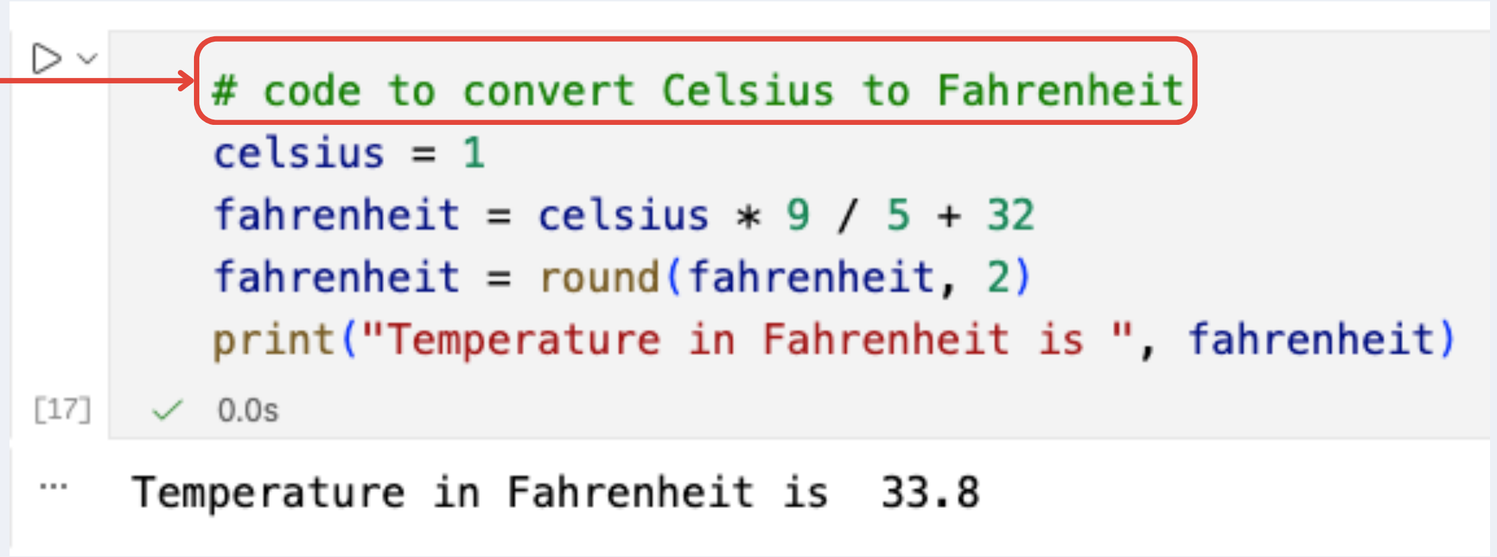
```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

comment, no effect in code



WORKING ON FORMULAS

declare a variable named celsius

set its value to 1

```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

- A variable is a named value (e.g. name, age, speed, area).
- We can assign a value to a variable using an assignment statement represented by the equal symbol (=).

WARNING:

- The symbol (=)means assignment (not equality).
  Assignment is not symmetric.
  Python assignment always go left.

WORKING ON FORMULAS

- Why use variables?

```
1 print("Juan is a student")
2 print("Juan is 16 years old")
3 print("Juan likes programming")
```

```
Juan is a student
Juan is 16 years old
Juan likes programming
```

```
1 name = "Juan"
2 print(name, "is a student")
3 print(name, "is 16 years old")
4 print(name, "likes programming")
```

```
Juan is a student
Juan is 16 years old
Juan likes programming
```

```
1 name = "Joe"
2 print(name, "is a student")
3 print(name, "is 16 years old")
4 print(name, "likes programming")
```

```
Joe is a student
Joe is 16 years old
Joe likes programming
```

WORKING ON FORMULAS

declare a variable named celsius

set its value to 1

```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

evaluate the expression on the left-side of symbol =

then assign result to fahrenheit

```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

round off
fahrenheit
to two decimal digits

then assign result to the
same variable

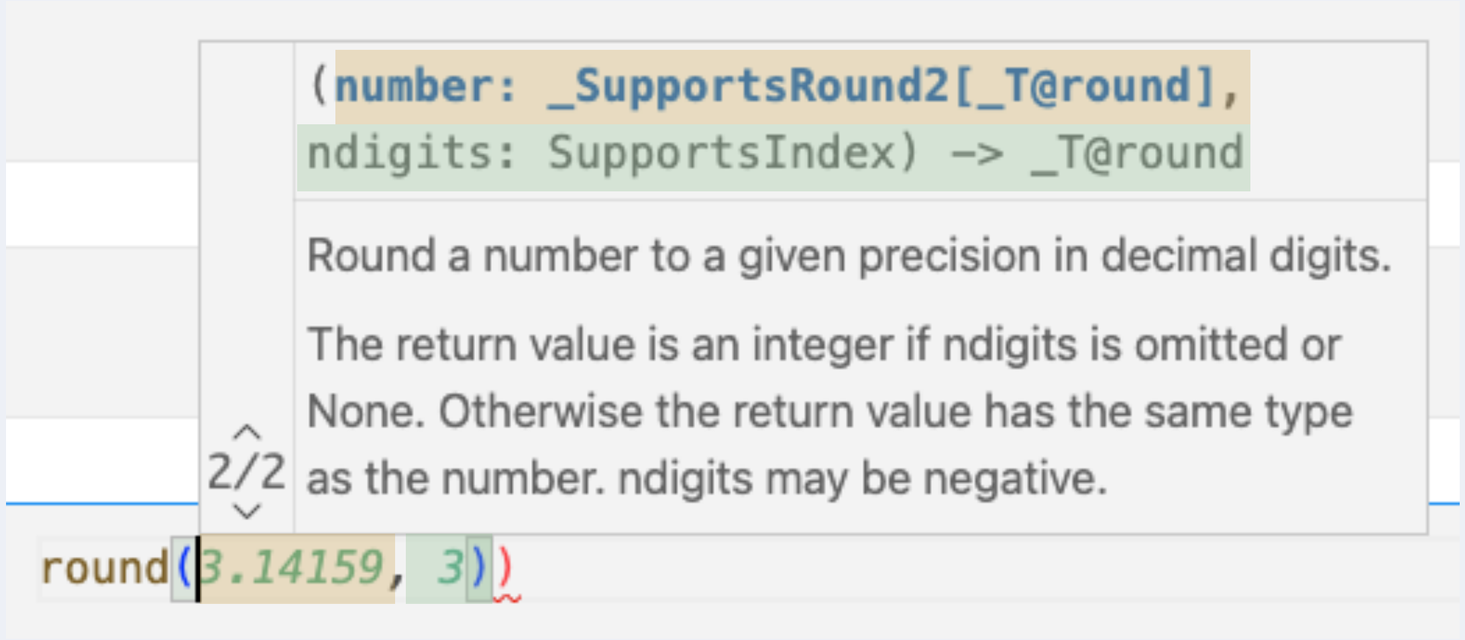
```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

If you want to know more info about a certain command, you can use the auto-complete feature of VS Code



WORKING ON FORMULAS

print some text followed by the value of fahrenheit

```
# code to convert Celsius to Fahrenheit
celsius = 1
fahrenheit = celsius * 9 / 5 + 32
fahrenheit = round(fahrenheit, 2)
print("Temperature in Fahrenheit is ", fahrenheit)
```

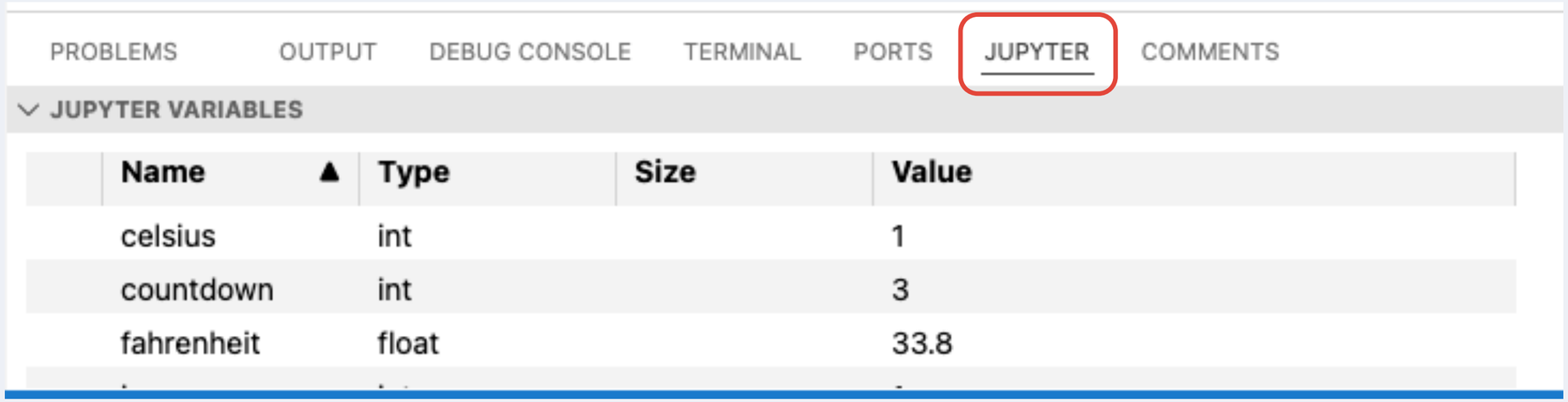
[17] ✓ 0.0s

```
Temperature in Fahrenheit is  33.8
```

WORKING ON FORMULAS

More on Variables

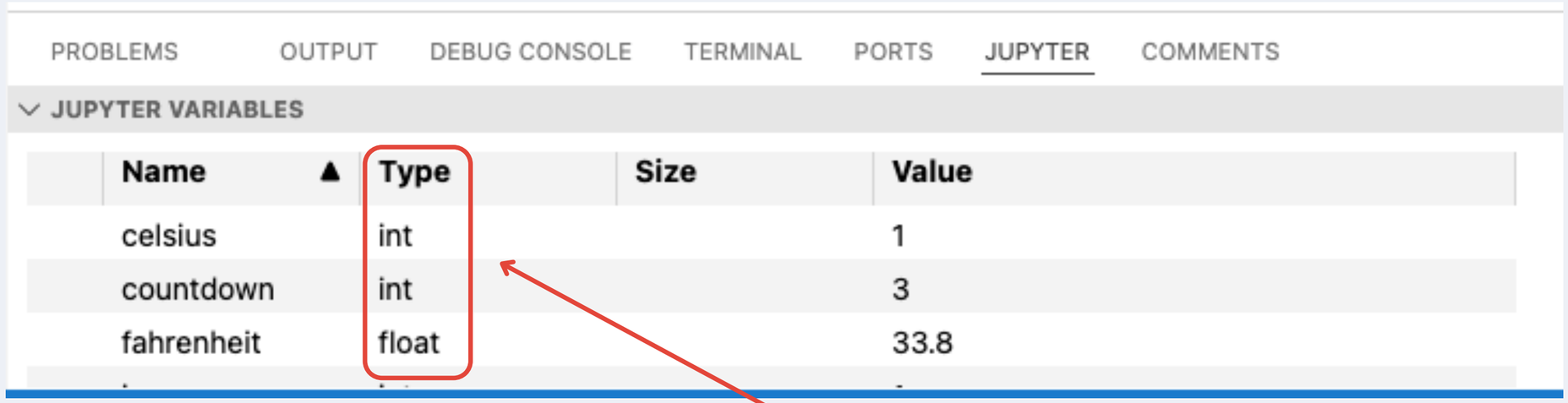
Under Panel, there is a tab called JUPYTER that keeps track of the variables declared in your notebook



WORKING ON FORMULAS

More on Variables

Under Panel, there is a tab called JUPYTER that keeps track of the variables declared in your notebook



indicates the data type of the variable
int means integer
float means a decimal number

CHOOSING A RANDOM NAME

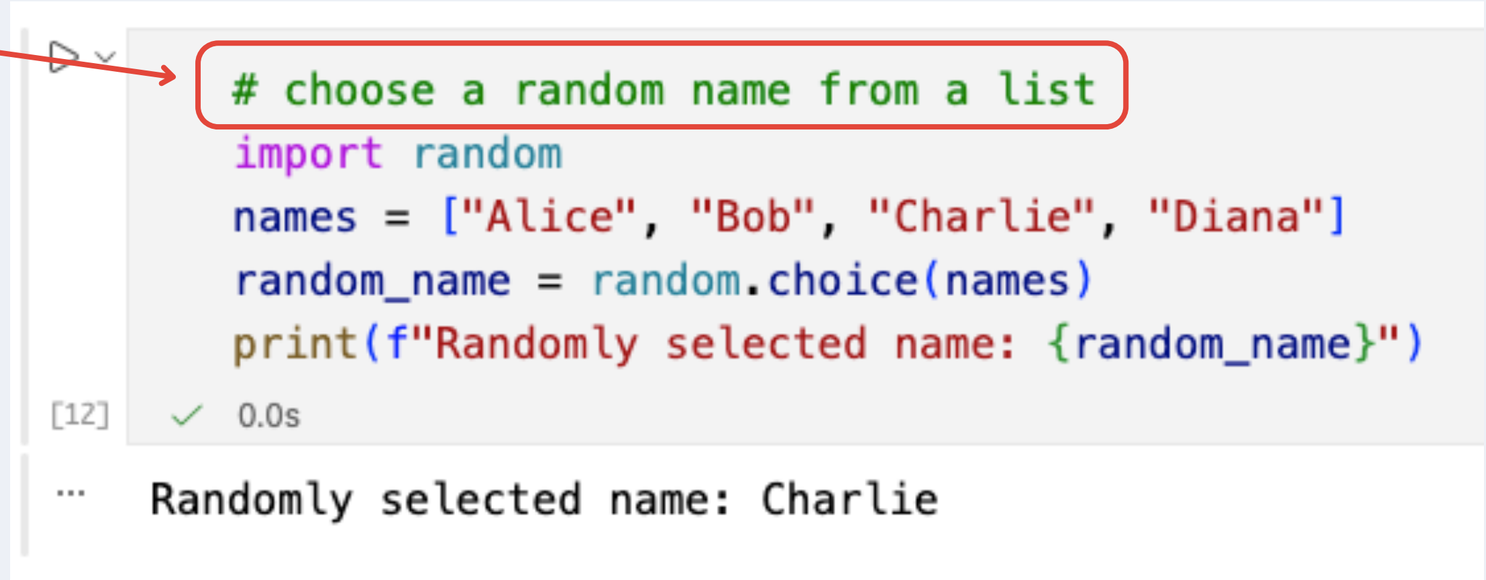
```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

[12] ✓ 0.0s

```
Randomly selected name: Charlie
```

CHOOSING A RANDOM NAME

comment, no effect in code



CHOOSING A RANDOM NAME

Calling the module random

```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

[12] ✓ 0.0s

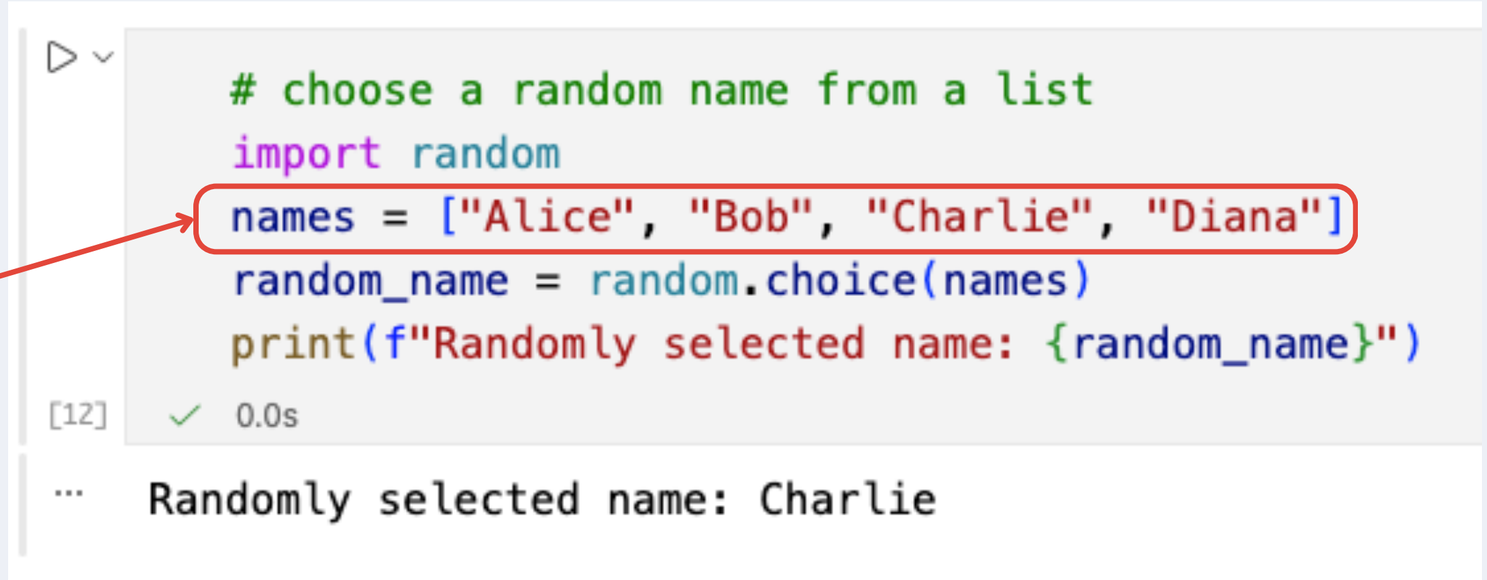
```
Randomly selected name: Charlie
```

CHOOSING A RANDOM NAME

- What is a module?
  - an external python file that we can import to our current python file
  - provides access to a collection of other commands (so-called functions)

CHOOSING A RANDOM NAME

The variable
names
is used to remember a collection of values (in particular, strings) represented as a so-called **list**



CHOOSING A RANDOM NAME

left side of symbol = performs the function choice within module random

the chosen string is assigned to random_name

```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

[12] ✓ 0.0s

```
Randomly selected name: Charlie
```

CHOOSING A RANDOM NAME

prints a string and the chosen name

```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

[12] ✓ 0.0s

```
Randomly selected name: Charlie
```

CHOOSING A RANDOM NAME

this is called a formatted-string (as marked by the f that precedes the string)

with formatted string, we can include variables inside the string

the variables are enclosed inside curly braces

```
# choose a random name from a list
import random
names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)
print(f"Randomly selected name: {random_name}")
```

[12] ✓ 0.0s

```
Randomly selected name: Charlie
```

CHOOSING A RANDOM NAME

```python
# choose a random name from a list
import time
import random

names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)

countdown = 5
# countdown timer before printing the random name
for remaining in range(countdown, 0, -1):
    print(f"Printing random name in {remaining}...")
    time.sleep(1)

print(f"Randomly selected name: {random_name}")
```

[3] ✓ 5.0s

```
Printing random name in 5...
Printing random name in 4...
Printing random name in 3...
Printing random name in 2...
Printing random name in 1...
Randomly selected name: Diana
```

CHOOSING A RANDOM NAME

We already have some idea what the other lines do, except this part

```
# choose a random name from a list
import time
import random

names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)

countdown = 5
# countdown timer before printing the random name
for remaining in range(countdown, 0, -1):
    print(f"Printing random name in {remaining}...")
    time.sleep(1)

print(f"Randomly selected name: {random_name}")
```

[3] ✓ 5.0s

```
Printing random name in 5...
Printing random name in 4...
Printing random name in 3...
Printing random name in 2...
Printing random name in 1...
Randomly selected name: Diana
```

CHOOSING A RANDOM NAME

Let’s use Copilot to help us understand this code chunk

```
# choose a random name from a list
import time
import random

names = ["Alice", "Bob", "Charlie", "Diana"]
random_name = random.choice(names)

countdown = 5
# countdown timer before printing the random name
for remaining in range(countdown, 0, -1):
    print(f"Printing random name in {remaining}...")
    time.sleep(1)

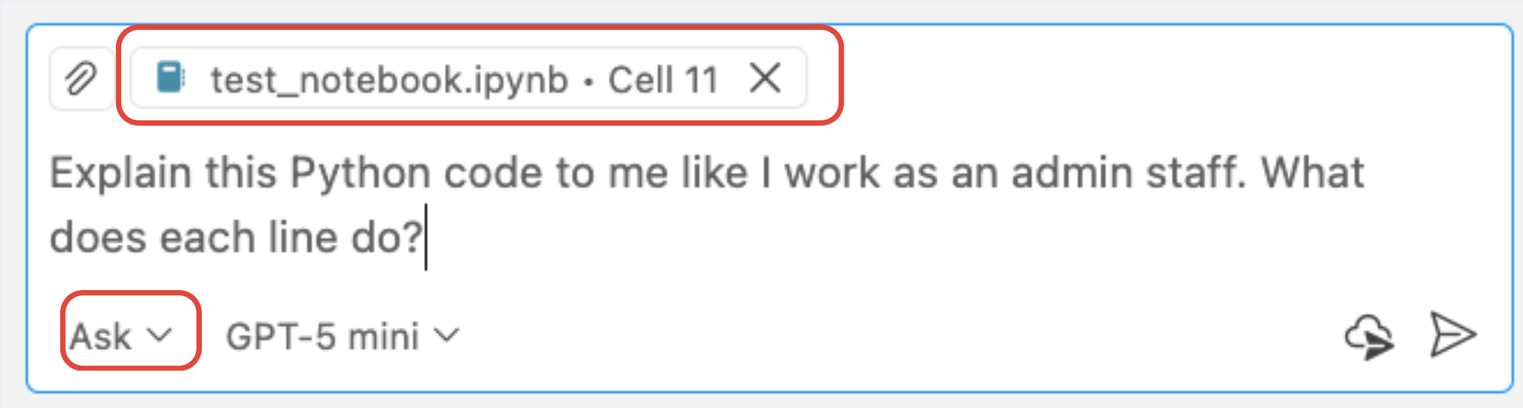
print(f"Randomly selected name: {random_name}")
```

[3] ✓ 5.0s

```
Printing random name in 5...
Printing random name in 4...
Printing random name in 3...
Printing random name in 2...
Printing random name in 1...
Randomly selected name: Diana
```

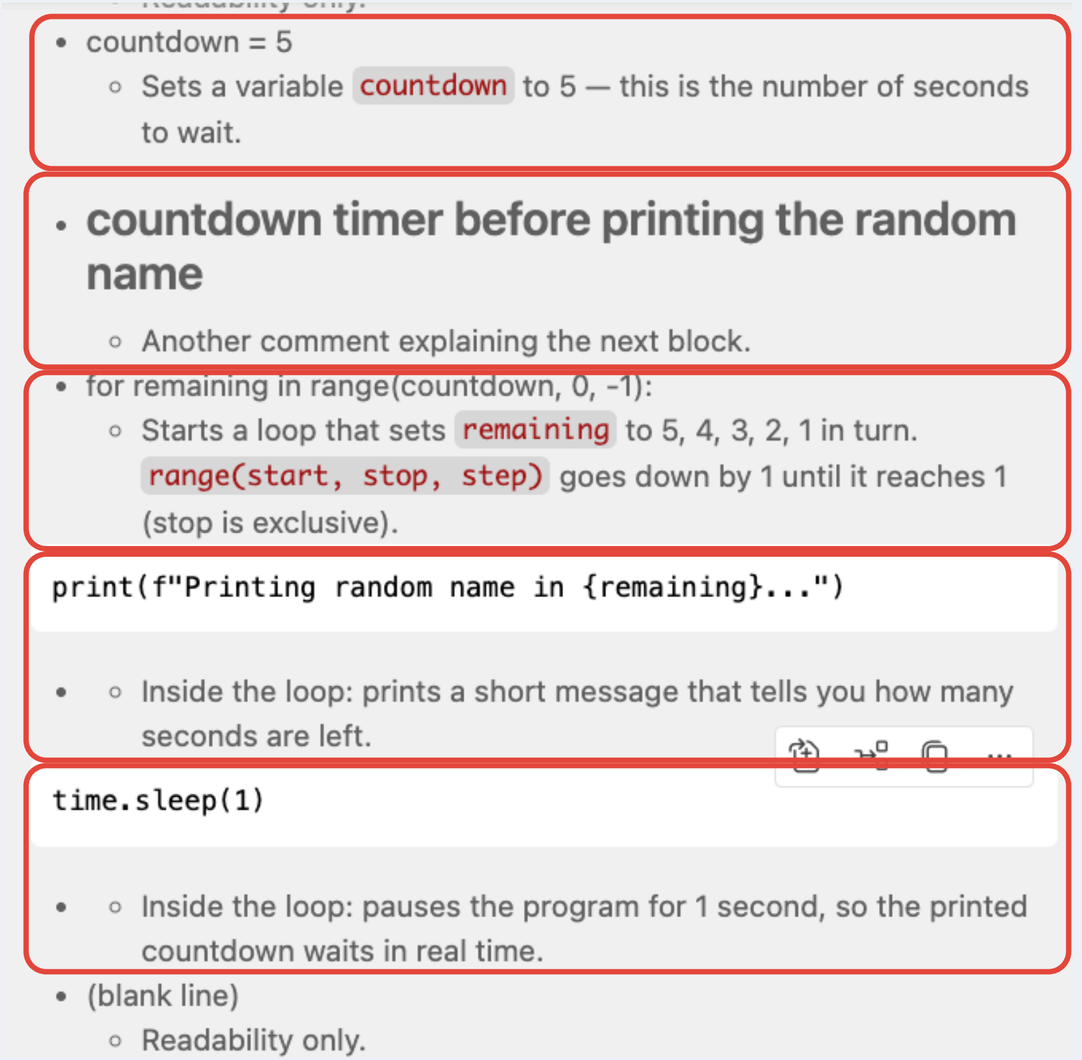
CHOOSING A RANDOM NAME

1. Set Chat to be in Ask mode.
2. Click to the target code cell.
3. Ask away.



CHOOSING A RANDOM NAME

1. Set Chat to be in Ask mode.
2. Click to the target code cell.
3. Ask away.



- countdown = 5
  - Sets a variable `countdown` to 5 — this is the number of seconds to wait.
- **countdown timer before printing the random name**
  - Another comment explaining the next block.
- for remaining in range(countdown, 0, -1):
  - Starts a loop that sets `remaining` to 5, 4, 3, 2, 1 in turn. `range(start, stop, step)` goes down by 1 until it reaches 1 (stop is exclusive).

```
print(f"Printing random name in {remaining}...")
```

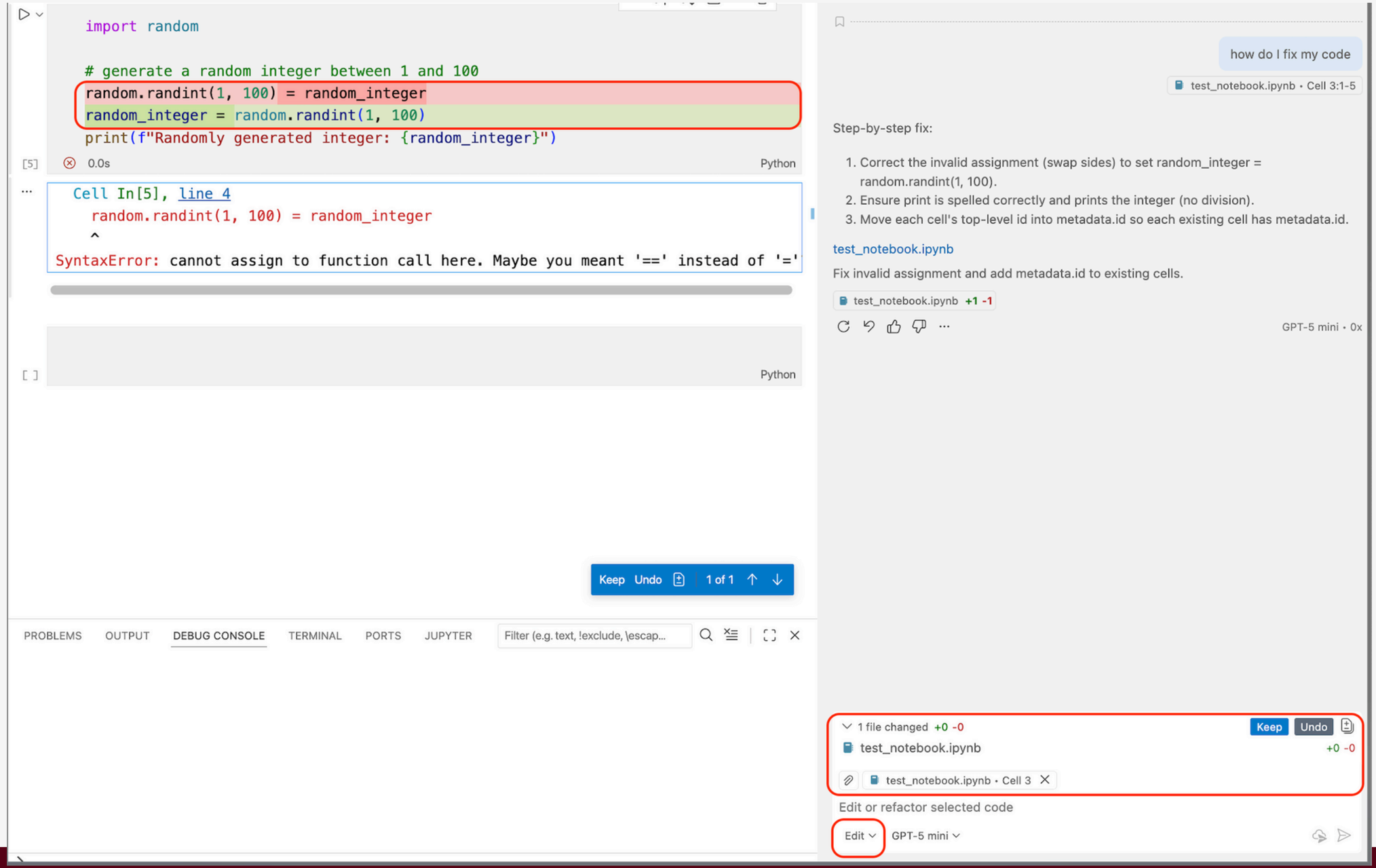
- Inside the loop: prints a short message that tells you how many seconds are left.

```
time.sleep(1)
```

- Inside the loop: pauses the program for 1 second, so the printed countdown waits in real time.
- (blank line)
  - Readability only.

TYPES OF ERRORS IN PYTHON

- Syntax Errors
  - indicate that Python didn’t understand the instruction or code provided
- Runtime Errors
  - indicate that Python understand the instruction or code, however, upon execution, a problem was encountered
- Semantic Errors
  - these are errors that Python cannot detect; the code may run smoothly, but the intended result is not reflected


import random
generate a random integer between 1 and 100
random.randint(1, 100) = random_integer
random_integer = random.randint(1, 100)
print(f"Randomly generated integer: {random_integer}")
[5]
0.0s
Python
Cell In[5], line 4
random.randint(1, 100) = random_integer
^
SyntaxError: cannot assign to function call here. Maybe you meant '==' instead of '='
[ ]
Python
Keep Undo 1 of 1
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS JUPYTER
Filter (e.g. text, !exclude, \escap...
how do I fix my code
test_notebook.ipynb • Cell 3:1-5
Step-by-step fix:
1. Correct the invalid assignment (swap sides) to set random_integer = random.randint(1, 100).
2. Ensure print is spelled correctly and prints the integer (no division).
3. Move each cell's top-level id into metadata.id so each existing cell has metadata.id.
test_notebook.ipynb
Fix invalid assignment and add metadata.id to existing cells.
test_notebook.ipynb +1 -1
GPT-5 mini • 0x
1 file changed +0 -0
Keep Undo
test_notebook.ipynb +0 -0
test_notebook.ipynb • Cell 3
Edit or refactor selected code
Edit
GPT-5 mini

```python
# get all even numbers from 1 to 50 and print each in word form
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
print(f"Even numbers from 1 to 50: {even_numbers}")
```

[7] ✓ 0.0s Python

```
Even numbers from 1 to 50: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 3
```

```python
# get all even numbers from 1 to 50 and print each in word form
even_numbers = [num for num in range(1, 51) if num % 2 == 0]
print(f"Even numbers from 1 to 50: {even_numbers}")
```

[7] ✓ 0.0s Python

```
Even numbers from 1 to 50: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20, 22, 24, 26, 28, 30, 3
```

```python
# get all even numbers from 1 to 50
# get all even numbers from 1 to 50 and print each in word form
def num_to_words(n):
    units = ['zero','one','two','three','four','five','six','seven','eight','nine'
             'eleven','twelve','thirteen','fourteen','fifteen','sixteen','seventee
    tens = ['','','twenty','thirty','forty','fifty']
    if n < 20:
        return units[n]
    else:
        t = n // 10
        u = n % 10
        return tens[t] + ('' if u == 0 else ' ' + units[u])

even_numbers = [num for num in range(1, 51) if num % 2 == 0]
print(f"Even numbers from 1 to 50: {even_numbers}")

for n in even_numbers:
    print(f"{n}: {num_to_words(n)}")
```

Keep Undo

] ✓ 0.0s Python

modify the code to print each number in word form

test_notebook.ipynb • Cell 4:1-4

Step-by-step solution

1. Update cell 4 (the code that builds even_numbers) to convert each number to words and print one per line. Add a small num_to_words helper that covers 1–50.
2. Ensure every existing cell has metadata.id (move the current top-level id into metadata.id).
3. Save the notebook.

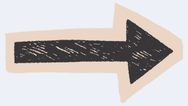
test_notebook.ipynb

Change cell 4 to print each even number in word form and add metadata.id to existing cells.

test_notebook.ipynb +1 -1

SOME NOTES IN PROMPTING

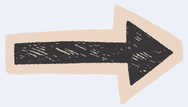
- Be specific
- Provide examples
- Ask for changes iteratively
- Ask for verification



Write a simple Python script that reads an Excel file called employee_hours.xlsx, adds up all hours in the ‘Hours Worked’ column, and prints the total.

SOME NOTES IN PROMPTING

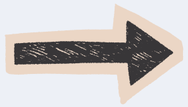
- Be specific
- Provide examples
- Ask for changes iteratively
- Ask for verification



"Here are two examples of input/output...
Now write a function that..."

SOME NOTES IN PROMPTING

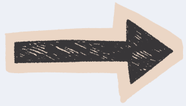
- Be specific
- Provide examples
- Ask for changes iteratively
- Ask for verification



- “add error handling”
- “turn this into a reusable function”
- add a check that warns me if the file is empty?

SOME NOTES IN PROMPTING

- Be specific
- Provide examples
- Ask for changes iteratively
- Ask for verification



- “List possible bugs or edge cases in this code.”
- “Suggest improvements.”

SUMMARY (SO FAR)

We learned about:

- some Python concepts commonly found in codes
- how to use Copilot to understand and fix codes

EXERCISES:

1. Create a “Hello, World!” program
2. Ask Copilot to greet all office members
3. Ask Copilot to generate code for “printing today’s date”
4. Ask Copilot to generate code for savings greetings to a file.

Download Guided Worksheet 01 at https://github.com/rbjuayong-ship-it/dxlabs/tree/main

WORKING WITH DATA USING COPILOT

OBJECTIVE: Learn some way to load and analyze data through codes generated by Copilot

A SAMPLE CSV FILE

	A	B	C	D
1	name	department	training_completed	
2	Alice Johnson	Human Resources	Yes	
3	Brian Smith	Information Technology	No	
4	Carla Martinez	Finance	Yes	
5	David Lee	Marketing	Yes	
6	Emily Chen	Operations	No	
7	Frank Turner	Information Technology	Yes	
8	Grace Patel	Finance	Yes	
9	Henry Adams	Human Resources	No	
10	Isabella Cruz	Marketing	Yes	
11	Jason Wright	Operations	Yes	
12	Karen Brooks	Finance	No	
13	Liam O'Connor	Information Technology	Yes	
14	Mia Thompson	Human Resources	Yes	
15	Noah Davis	Marketing	No	
16	Olivia Lewis	Operations	Yes	
17	Paul Robinson	Information Technology	No	
18	Quinn Foster	Finance	Yes	
19	Riley Morgan	Human Resources	Yes	
20	Sophia Bennett	Marketing	Yes	
21	Thomas Walker	Operations	No	

```
name,department,training_completed
Alice Johnson,Human Resources,Yes
Brian Smith,Information Technology,No
Carla Martinez,Finance,Yes
David Lee,Marketing,Yes
Emily Chen,Operations,No
Frank Turner,Information Technology,Yes
Grace Patel,Finance,Yes
Henry Adams,Human Resources,No
Isabella Cruz,Marketing,Yes
Jason Wright,Operations,Yes
Karen Brooks,Finance,No
Liam O'Connor,Information Technology,Yes
Mia Thompson,Human Resources,Yes
Noah Davis,Marketing,No
Olivia Lewis,Operations,Yes
Paul Robinson,Information Technology,No
Quinn Foster,Finance,Yes
Riley Morgan,Human Resources,Yes
Sophia Bennett,Marketing,Yes
Thomas Walker,Operations,No
```

SAMPLE CODE TO LOAD CSV

Copilot suggestion

```python
# load "training.csv" data in and print each row
import csv
with open('training.csv', mode='r') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        print(row)
```

[5] ✓ 0.0s Python

```
['name', 'department', 'training_completed']
['Alice Johnson', 'Human Resources', 'Yes']
['Brian Smith', 'Information Technology', 'No']
['Carla Martinez', 'Finance', 'Yes']
['David Lee', 'Marketing', 'Yes']
['Emily Chen', 'Operations', 'No']
```

SAMPLE CODE TO LOAD CSV

```python
# load "training.csv" data in and print each row
import csv

path = 'training.csv'
with open(path, mode='r', encoding='utf-8-sig', newline='') as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row)
```

[13] ✓ 0.0s

```
{'name': 'Alice Johnson', 'department': 'Human Resources', 'training_completed': 'Yes'}
{'name': 'Brian Smith', 'department': 'Information Technology', 'training_completed': 'No'}
{'name': 'Carla Martinez', 'department': 'Finance', 'training_completed': 'Yes'}
{'name': 'David Lee', 'department': 'Marketing', 'training_completed': 'Yes'}
{'name': 'Emily Chen', 'department': 'Operations', 'training_completed': 'No'}
{'name': 'Frank Turner', 'department': 'Information Technology', 'training_completed': 'Yes'}
{'name': 'Grace Patel', 'department': 'Finance', 'training_completed': 'Yes'}
{'name': 'Henry Adams', 'department': 'Human Resources', 'training_completed': 'No'}
{'name': 'Isabella Cruz', 'department': 'Marketing', 'training_completed': 'Yes'}
{'name': 'Jason Wright', 'department': 'Operations', 'training_completed': 'Yes'}
{'name': 'Karen Brooks', 'department': 'Finance', 'training_completed': 'No'}
{'name': "Liam O'Connor", 'department': 'Information Technology', 'training_completed': 'Yes'}
```

another Copilot suggestion

SAMPLE CODE TO PROCESS CSV

Copilot suggestion

but is slightly wrong

```python
# load "training.csv" data in and count names who completed the training
import csv
completed_count = 0
with open('training.csv', mode='r') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        if row[1].lower() == 'yes':
            completed_count += 1
print(f"Number of people who completed the training: {completed_count}")
```

✓ 0.0s Python

```
Number of people who completed the training: 0
```

SAMPLE CODE TO PROCESS CSV

corrected

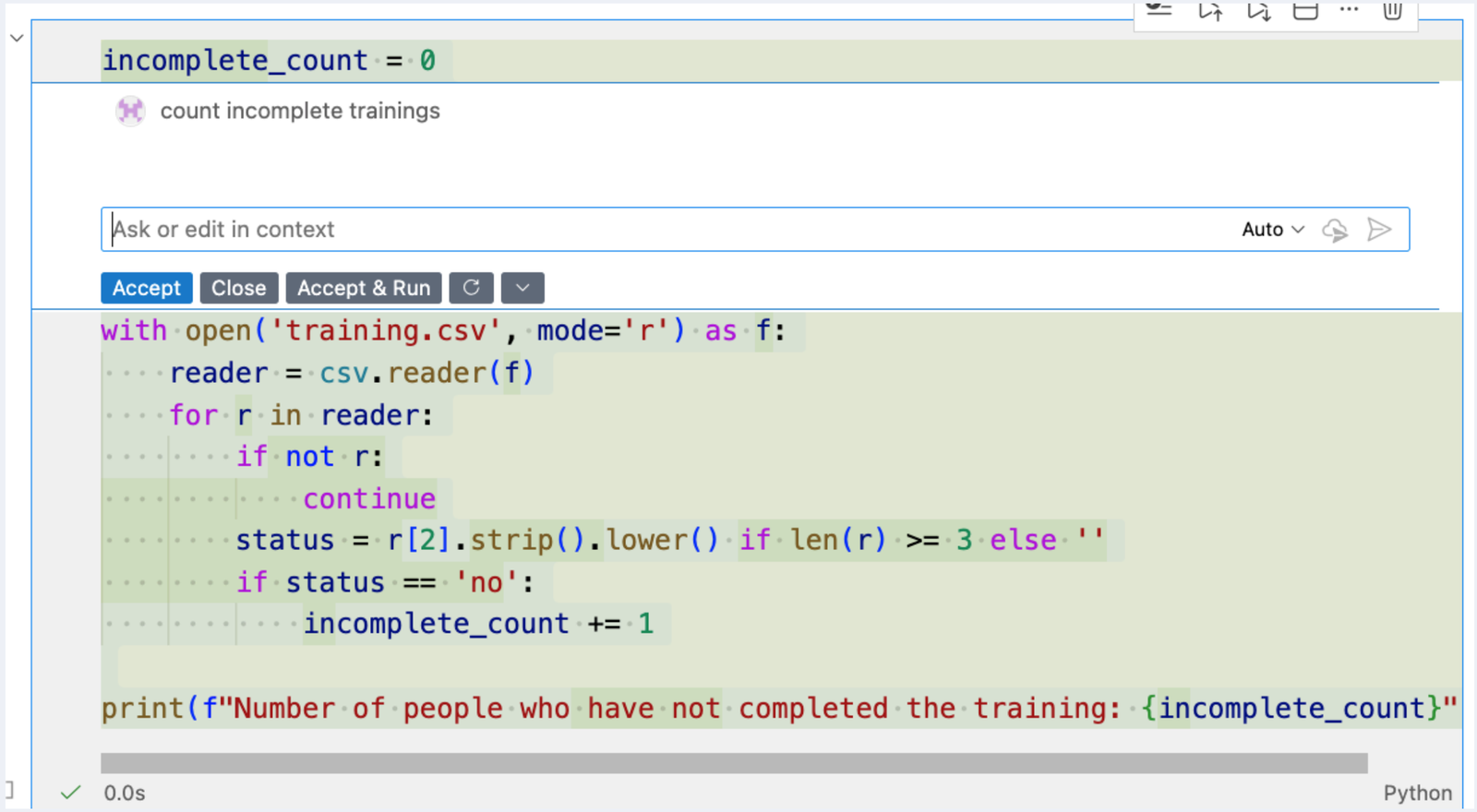
Copilot suggestion

```python
# load "training.csv" data in and count names who completed the training
import csv
completed_count = 0
with open('training.csv', mode='r') as file:
    csv_reader = csv.reader(file)
    for row in csv_reader:
        if row[2].lower() == 'yes':
            completed_count += 1
print(f"Number of people who completed the training: {completed_count}")
```

✓ 0.0s Python

```
Number of people who completed the training: 13
```

SAMPLE CODE TO PROCESS CSV



Using inline chat to modify the code

SAMPLE CODE TO PROCESS CSV

```
incomplete_count = 0
with open('training.csv', mode='r') as f:
    reader = csv.reader(f)
    for r in reader:
        if not r:
            continue
        status = r[2].strip().lower() if len(r) >= 3 else ''
        if status == 'no':
            incomplete_count += 1

print(f"Number of people who have not completed the training: {incomplete_count}"
```

[7] ✓ 0.0s Python

```
Number of people who have not completed the training: 7
```

Using inline chat to modify the code

TRY IT YOURSELF!

Ask AI to modify the code in order to:

1. Show names of people needing training
2. Group by department
3. Change the names to uppercase and print the result in a new file

TRY IT YOURSELF!

```python
# group by department and count completed vs incomplete (no/other)
dept_counts = {}

with open('training.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    for i, row in enumerate(reader):
        if not row:
            continue
        # skip header if present
        if i == 0 and any(h.strip().lower() in ('name', 'department', 'status') for h in row):
            continue

        dept = row[1].strip() if len(row) > 1 and row[1].strip() else 'Unknown'
        status = row[2].strip().lower() if len(row) > 2 else ''

        completed = status in ('yes', 'y', 'completed', 'complete', 'true', '1')
        counts = dept_counts.setdefault(dept, {'completed': 0, 'incomplete': 0, 'total': 0})

        if completed:
            counts['completed'] += 1
        else:
            counts['incomplete'] += 1
        counts['total'] += 1

# print results sorted by department name
for dept in sorted(dept_counts):
    c = dept_counts[dept]
    print(f"{dept}: {c['completed']} completed, {c['incomplete']} incomplete (total {c['total']})")
```

[8] ✓ 0.0s

```
Finance: 3 completed, 1 incomplete (total 4)
Human Resources: 3 completed, 1 incomplete (total 4)
Information Technology: 2 completed, 2 incomplete (total 4)
Marketing: 3 completed, 1 incomplete (total 4)
Operations: 2 completed, 2 incomplete (total 4)
```

TRY IT YOURSELF!

```
# load "training.csv" data in and modify the names to uppercase, then print each row to a new file "training_uppercase.csv"
import csv
with open('training.csv', mode='r', newline='', encoding='utf-8') as infile, \
     open('training_uppercase.csv', mode='w', newline='', encoding='utf-8') as outfile:
    csv_reader = csv.reader(infile)
    csv_writer = csv.writer(outfile)

    for row in csv_reader:
        if row:  # ensure the row is not empty
            row[0] = row[0].upper()  # convert the name (first column) to uppercase
        csv_writer.writerow(row)
```

TRY IT YOURSELF!

training_uppercase.csv

```
NAME,department,training_completed
ALICE JOHNSON,Human Resources,Yes
BRIAN SMITH,Information Technology,No
CARLA MARTINEZ,Finance,Yes
DAVID LEE,Marketing,Yes
EMILY CHEN,Operations,No
FRANK TURNER,Information Technology,Yes
GRACE PATEL,Finance,Yes
HENRY ADAMS,Human Resources,No
ISABELLA CRUZ,Marketing,Yes
JASON WRIGHT,Operations,Yes
KAREN BROOKS,Finance,No
LIAM O'CONNOR,Information Technology,Yes
MIA THOMPSON,Human Resources,Yes
NOAH DAVIS,Marketing,No
OLIVIA LEWIS,Operations,Yes
PAUL ROBINSON,Information Technology,No
QUINN FOSTER,Finance,Yes
RILEY MORGAN,Human Resources,Yes
SOPHIA BENNETT,Marketing,Yes
THOMAS WALKER,Operations,No
```

CULMINATION ACTIVITY

CO-PILOT YOUR OWN CSV FILE

Use co-pilot in VSCode to generate code that will read a .csv file of your own dataset and automate a **simple task** on that file.

CULMINATION ACTIVITY

CO-PILOT YOUR OWN CSV FILE

1. Share to the group a short description of the dataset.
2. Talk about the goal or task you want to perform on your dataset
3. Share the result of co-pilot and describe your insights regarding the generated code. How helpful was it? Was it able to do correctly what you want it to be.

WRAP-UP, REFLECTION, AND PREVIEW

OBJECTIVE: Reinforce concepts and connect to future sessions

RECAP

We learned and experienced:

1. Basic navigation on Visual Studio Code
2. How to work on Jupyter notebooks
3. How to use Github Copilot
4. Study some simple Python codes
5. How to use Copilot to understand codes
6. How to use Copilot to process some data

END OF SLIDES